Cross-Domain Recommender System

Domains: *Movies → Games*

IRS Project Report

A systematic experimental study of cross-domain recommendation using the Amazon Reviews 2023 dataset. Leveraging movie preferences to recommend video games.

Vo Minh Khoi - A0339229W

1. Executive Summary

Abstract. We set out to investigate cross-domain recommendation (CDR) for transferring user preferences from movies to video games on the Amazon Reviews 2023 dataset. The central question is: can a user's movie history improve game recommendations, if yes, under which data condition? We run a series of systematic comparison benchmarks across three model families — single-domain collaborative filtering (MF-BPR, NCF, LightGCN), cross-domain transfer (CMF, EMCDR, PTUPCDR), and content-based (SBERT-CDR) — evaluated head-to-head on multiple user subgroups.

Key Findings

- **User Overlap is Critical**: CDR models require high user overlap to outperform single-domain baselines. On the 100%-overlap subgroup, CDR models recovered a lift of +41% (EMCDR) to +750% (CMF) over the low-overlap baseline.
- **Cold-Start Effectiveness**: At true cold-start (zero game history, L5), every CDR model beat every single-domain collaborative model.
- **Co-occurrence Reranking**: A training-free co-occurrence reranking layer improved nearly every base model (5–49% lift on LLO) and successfully rescued single-domain models at cold-start (+289% for LightGCN).
- **Simple models can be unexpectedly powerful**: the training-free SBERT-CDR content-based model performs as effectively as other more complicated models.

2. Introduction

2.1 Background and Problem Statement

Most production recommendation systems are single-domain: they recommend items based solely on a user's interaction history within that single domain. Cross-Domain Recommendation (CDR) relaxes this constraint — it transfers knowledge from a source domain (movies) to a target domain (games), which is especially valuable at cold-start: when a user has little or no target-domain rating history, CDR can still recommend target items by leveraging their source-domain preferences.

Market Research

The entertainment industry is actively shifting from single-domain apps toward multi-domain platforms. Netflix has expanded from video streaming into mobile games, Amazon sells across every consumer category from a single storefront, and Spotify has merged music and podcast discovery into one unified feed. The industry accordingly has clear demand for multi-domain recommenders that can rank across an entire platform catalog, motivated by three measurable business outcomes — higher retention, stronger cross-sell, and happy-surprise discovery (Figure 1).

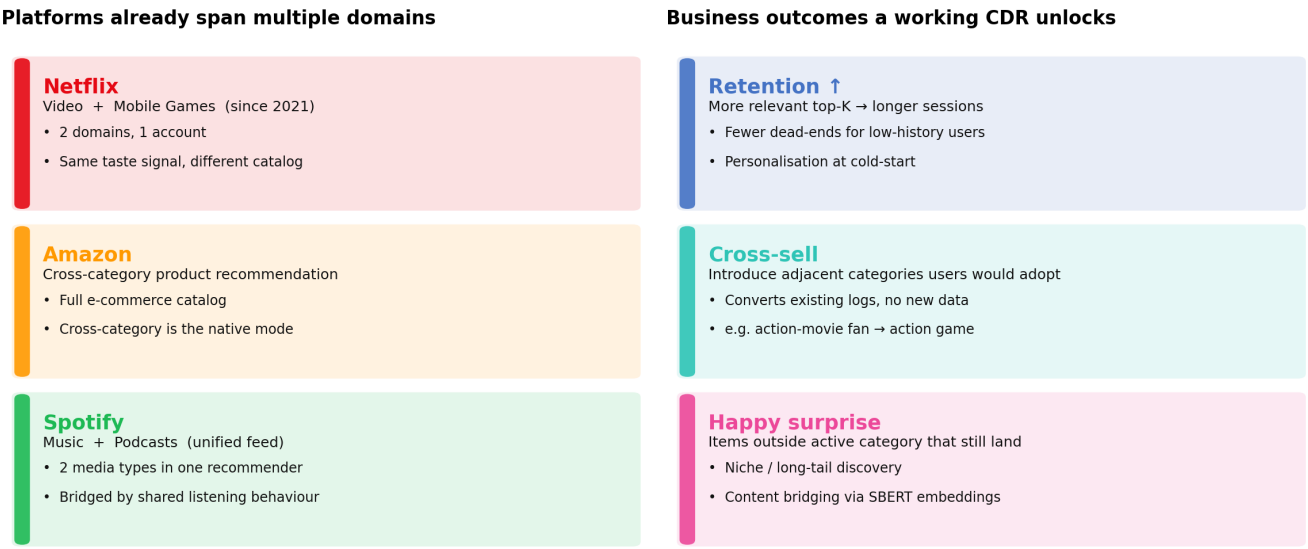


Figure 1: Platforms are already multi-domain; a working CDR converts existing logs into retention, cross-sell, and happy-surprise outcomes.

Why this matters: Untapped Cross-Domain Preference Signal

Modern consumers engage with entertainment across multiple content categories — films, television, games, music, podcasts — often within a single session and almost always within a single platform account. Industry recommender systems, by contrast, have historically been architected on a per-domain basis: each model observes user behavior within the boundaries of its own domain and optimizes ranking against that domain alone. This is an engineering simplification, not a reflection of user behavior, and it leaves a large share of each user's observable preference signal untapped:

- **Overlapping interests**: action-movie fans often play action games; fantasy-film viewers often play RPGs. Single-domain models cannot use this link.
- **Missed signals**: source-domain behaviour (movies) is dense for most users; ignoring it throws away the best available preference data for a target-domain (games) cold-start.
- **Cross-sell gap**: without a cross-domain ranker, the platform cannot surface adjacent categories a user would naturally adopt.

Problem statement

This project studies movie-to-game recommendation as a concrete cross-domain setting. Given users with movie-rating histories, we ask whether source-domain preferences can improve target-domain game recommendations, especially for users with sparse or zero game history. The final deliverable is an experimentally grounded routing rule that selects an appropriate recommendation strategy based on each user's target-domain history depth and item popularity profile.

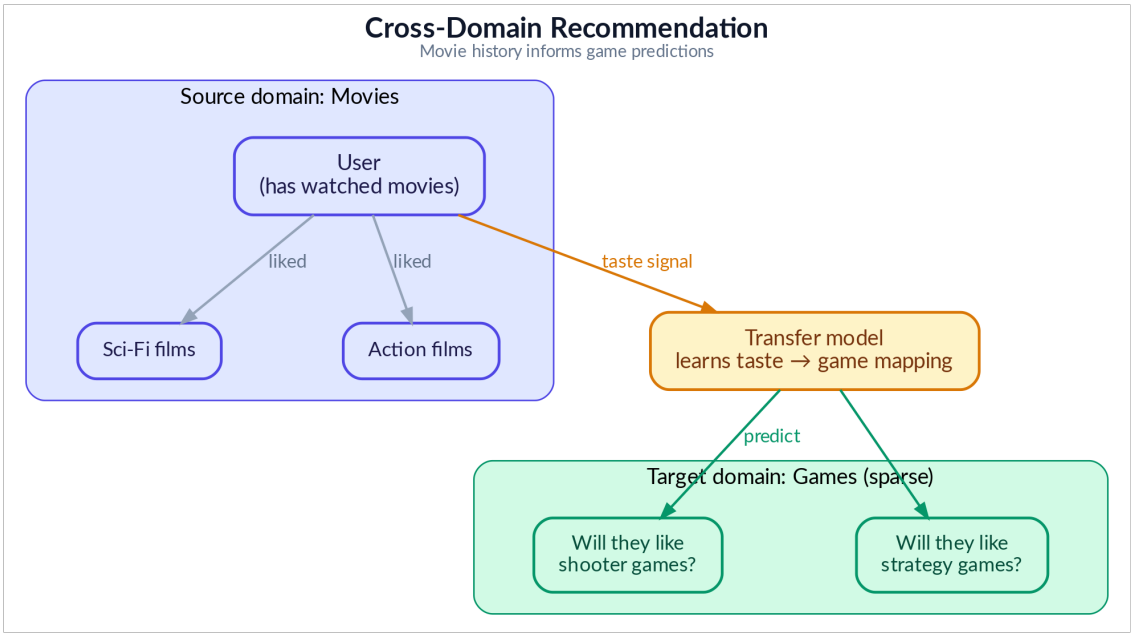


Figure 2: Cross-domain recommendation concept — transferring movie preferences to game recommendations

2.2 Research Methodology and Experiment Design

We present the experiments as a clean, progressive lesson series — but the path to this structure was **far less linear**. This section describes our actual research process, the challenges encountered, and how the final experiment design emerged from extensive exploratory works.

Phase 1: Model Selection and Infrastructure Setup

We began by studying the internal mechanisms of cross-domain recommendation algorithms — how cross domain models learn to map user preferences across domains, and how they differ from single-

domain approaches that rely purely on within-domain interactions. This informed our model selection: three cross domain recommendation CDR) architectures (joint factorization, global mapping, personalized mapping), three single domain recommendation (SDR) baselines (matrix factorization, neural, graph-based) to ensure fair comparison across model families.

Here are the rationales for choosing each set of algorithms:

Single-Domain Recommendation (SDR):

We picked these models to cover the three major collaborative filtering paradigms, trained only on game rating data, to serve as strong competitive baselines for the Cross-Domain Recommendation (CDR) models.

- **MF-BPR (Matrix Factorization with Bayesian Personalized Ranking):** Chosen as a simple, well-understood, and fast-to-train baseline that captures linear feature interactions.
- **NCF (Neural Collaborative Filtering):** Selected to incorporate neural networks (MLPs) that can capture complex, non-linear patterns and subtle interactions that simple dot products (like in MF-BPR) cannot.
- **LightGCN (Graph Convolution):** LightGCN was included as a strong graph-based collaborative filtering baseline, widely used for implicit-feedback recommendation and expected to perform well when sufficient target-domain interactions are available.

Cross-Domain Recommendation (CDR):

We picked these models to cover the different ways knowledge can be transferred from the source domain (movies) to the target domain (games).

- **CMF (Collective Matrix Factorization):** Chosen as the simplest CDR architecture, representing **joint factorization**. It attempts to capture transferable traits by forcing a single, shared user profile to work for both domains.
- **EMCDR (Cross-Domain Embedding Mapping):** Represents a **global mapping** approach. It trains a single translator (a mapping network) to convert specialized movie user embeddings into game space, making it especially powerful at cold-start where a user has no game history.
- **PTUPCDR (Personalized Transfer with Experts):** Represents a **personalized mapping** approach. It enhances EMCDR by using a Mixture of Experts architecture to provide a personalized translation rule for each user.

Content-Based and Post-Processing:

These approaches were **not part of the initial algorithm selection** but were discovered during **Phase 4** (Discovering Complementary Approaches) of our exploratory work, proving **simpler** approaches yet unexpectedly **powerful** for specific recommendation regimes. They were subsequently included to address the known limitations of collaborative models, particularly for cold-start users.

- **SBERT-CDR (Content-Based Semantic Matching):** Used to assess content-based embeddings as a cross-domain bridge. We picked it because it is the training-free model that can recommend items effectively and works for brand-new users who have no rating history.

- **Co-occurrence Reranking:** Selected as a universal, training-free **post-processing re-ranking layer** to capture obvious cross-domain behavioral patterns. The underlying idea is similar to Market Basket Analysis taught in the course. It provides a model-agnostic score boost, improving every base model (5–49% lift), and successfully increased single-domain models at cold-start.

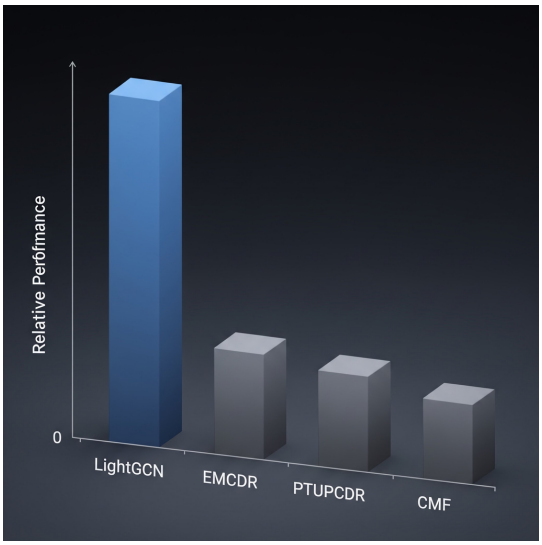
We then built the base infrastructure: **a data processing pipeline** (from raw and high sparsity Amazon reviews to filtered and better-dense user's reviews and item's review), **a standardized evaluation framework** (Recall, NDCG at K = 10), and **a benchmarking setup** that ensures all models are evaluated on identical data splits. All models were implemented using publicly available libraries (RecBole, RecBole-CDR, PyTorch, scikit-learn,...) to avoid re-inventing the wheel and ensure reproducibility.

We utilized AI coding tool assistants throughout the development of our system. AI-assisted coding tools were used to accelerate implementation. All generated code was manually reviewed, tested, and validated before inclusion in the final system. All AI usages were fully declared in Appendix E of the report.

Phase 2: Exploratory Benchmarking and the CDR Performance Disappointment

With the infrastructure in place, we ran extensive benchmarks comparing single-domain and cross-domain models across different data configurations. The initial results were **consistently discouraging** for CDR: LightGCN (a single-domain graph model) outperformed all CDR models in nearly every setting — including on user subgroups with rich movie history (>10 ratings) but sparse game history (3–5 ratings), precisely the regime where CDR is expected to excel.

We spent considerable effort trying different data configurations, different user subgroup filters, and hyperparameter settings. Unfortunately, the effort failed to increase the CDR performance. CDR models consistently failed to demonstrate a clear advantage over the single-domain baseline.



This raised a fundamental question: is the movie-to-game transfer signal in the Amazon dataset simply too weak for CDR to exploit? At one stage, these results raised doubts about whether movie-to-game transfer was measurable in the Amazon dataset. However, rather than concluding that CDR was ineffective, we revisited the assumptions in prior CDR studies and identified a **mismatch** between our initial data setup and the conditions under which CDR is expected to succeed.

Phase 3: Revisiting CDR Theory and Finding the Right Data Regime

As we mentioned, rather than concluding prematurely that CDR is ineffective — a conclusion contradicted by a large body of published research — we returned to the literature. Published CDR papers are evaluated under specific data conditions that we had not yet replicated: high user overlap between domains, rich source-domain history, and sparse or absent target-domain history.

> **Key Insight**
>
> The general-population Amazon dataset has only ~5.8% user overlap between movies and games — far too low for CDR mapping functions to learn from. Once we filtered to 100% overlap users and, crucially, tested on cold-start users with zero game history, CDR models finally outperformed their single-domain counterparts significantly. The algorithms were not broken — the data regime was wrong.

Phase 4: From Exploration to Structured Lessons

With all findings in hand, we designed the final lesson series to present our discoveries as a progressive and clear sequence. Each lesson isolates one experimental variable and builds on the previous lesson's conclusion. This structure was designed retrospectively — the actual research process involved **many more experiments**, several of which produced null or even negative results and thus were not included.

For example, in one experiment, we hypothesized that removing long-tail movie items with very few ratings would produce denser, higher-quality source embeddings and improve CDR transfer. We tested more rigorous **Movie genres filtering, overlap-user item filtering, and stricter movie popularity thresholds**. But the result was largely **null** — filtering hurt both single-domain LightGCN (smaller training catalog) and produced mixed results for CDR models.

> **Key Insight**
>
> So the exploration journey was not a linear, all happy-ending path, but a rather challenging and filled-with-roadblocks exploratory path.
>
> In summary, our research methodology was deliberately exploratory: we ran a broad set of experiments, identified which factors genuinely drive CDR effectiveness (and which do not), and then distilled the findings into a curated lesson series that progresses from simple to complex.

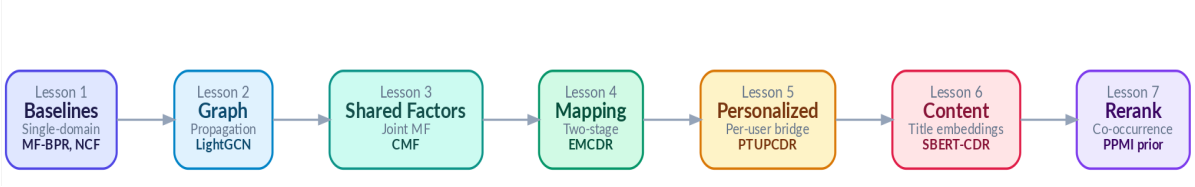


Figure 3: From Many Exploratory Experiments to 7 Curated Lessons

3. Data

3.1 Data Source

We use the Amazon Reviews 2023 dataset, a large-scale benchmark containing user reviews and ratings across multiple product categories. We focus on two domains: Movies and TV (user ratings of movies, DVDs, and streaming content) and Video Games (user ratings of video games across all platforms). You can download the data from these urls:

Movies & TV (target=movie):

- https://mcauleylab.ucsd.edu/public_datasets/data/amazon_2023/raw/review_categories/Movies_and_TV.jsonl.gz
- https://mcauleylab.ucsd.edu/public_datasets/data/amazon_2023/raw/meta_categories/meta_Movies_and_TV.jsonl.gz

Video Games (target=game):

- https://mcauleylab.ucsd.edu/public_datasets/data/amazon_2023/raw/review_categories/Video_Games.jsonl.gz
- https://mcauleylab.ucsd.edu/public_datasets/data/amazon_2023/raw/meta_categories/meta_Video_Games.jsonl.gz

3.2 Data Structure

The pipeline ingests four gzipped JSONL files from the Amazon Reviews 2023 dataset — review and metadata streams for the Movies & TV and Video Games domains — and emits a unified set of parquet artifacts that downstream training, evaluation, and serving all read from.

Raw inputs (Amazon Reviews 2023)

File	Domain	Format
movies_reviews_2023.jsonl.gz	Movies & TV	gzipped JSONL, one review per line
games_reviews_2023.jsonl.gz	Video Games	gzipped JSONL, one review per line
movies_meta_2023.jsonl.gz	Movies & TV	gzipped JSONL, one item per line
games_meta_2023.jsonl.gz	Video Games	gzipped JSONL, one item per line

Rating record schema

Field	Type	Description
user_id	string	Reviewer / user identifier
item_id	string	Amazon parent_id — canonical item identifier
rating	float	Star rating, 1–5
timestamp	datetime	When the review was posted; used for leave-last-out splitting
review_text	text	Free-form review body, truncated to 5000 chars
domain	string	"movies" or "games" — added at parse time

Item metadata schema

Field	Type	Description
external_id	string	ID — the universal key shared by DB, single-domain, and CDR layers
domain	string	"movies" or "games"
title	string	Item title (≤ 512 chars)
description	text	Long-form description (≤ 2000 chars); fed to SBERT
categories	string	Comma-joined Amazon category path; used to filter actual games
main_category	string	Top-level Amazon category
image_url	string	Hi-res product image URL surfaced in the demo UI

Processed parquet outputs

File	Granularity	Key columns	Purpose
ratings.parquet	one row per user–item interaction	user_id, item_id, rating, timestamp, domain	Training and evaluation interactions for both domains
movies.parquet	one row per movie item	external_id, title, description, categories, …	Movie-domain item catalog
games.parquet	one row per game item	external_id, title, description, categories, …	Game-domain item catalog
dataset_metadata.json	single file	processing parameters, item / user counts	Reproducibility metadata for each pipeline run

3.3 Data Pipeline

The data processing pipeline is responsible for converting the raw, sparse Amazon Reviews 2023 data into the standardized, dense, and implicit subgroups required for model training and evaluation.

Here is the table summarizing the key steps and their justifications:

Pipeline Step	Description/Value
Data Ingestion	Parse Amazon Reviews 2023 dataset (Movies/TV as source, Video Games as target). • Movie Ratings • Game Ratings • Movie Metadata • Game Metadata
Data Filtering	For Game items, filter out items that are not real Games by filter out items without `Games` category For example:

	• Xbox Gift Card, PS4 controllers… → removed because these items do not have `Games` in its category field
Data Merging	Merge items that are essentially same games/movies but different platforms by detecting `[text]` pattern For example: • *The Avengers[Blue Ray]* and *The Avengers[Digital]* → Remove [Blue Ray] and [Digital] → Merge to ***The Avengers***
Implicit Ratings Conversion	Ratings on a 1–5 star scale are converted to implicit feedback: ratings >= 4 stars are treated as positive interactions (POSITIVE_THRESHOLD = 4).
Density Filtering	Application of k-core filtering to both items and users to increase rating density: • Items must have at least >= X ratings • Users must have at least >= Y ratings X and Y depend on the experiments(vary to create different user sub groups)
Store In Files	Processed data stored as Parquet files for training, and final embeddings exported as NumPy arrays.
Split User Subgroups	Creation of specialized subgroups (Raw Unfiltered, Source-Rich, Cold Start) by applying specific filters (k-core, density, and overlap thresholds) to isolate experimental variables for lessons L1 through L7.

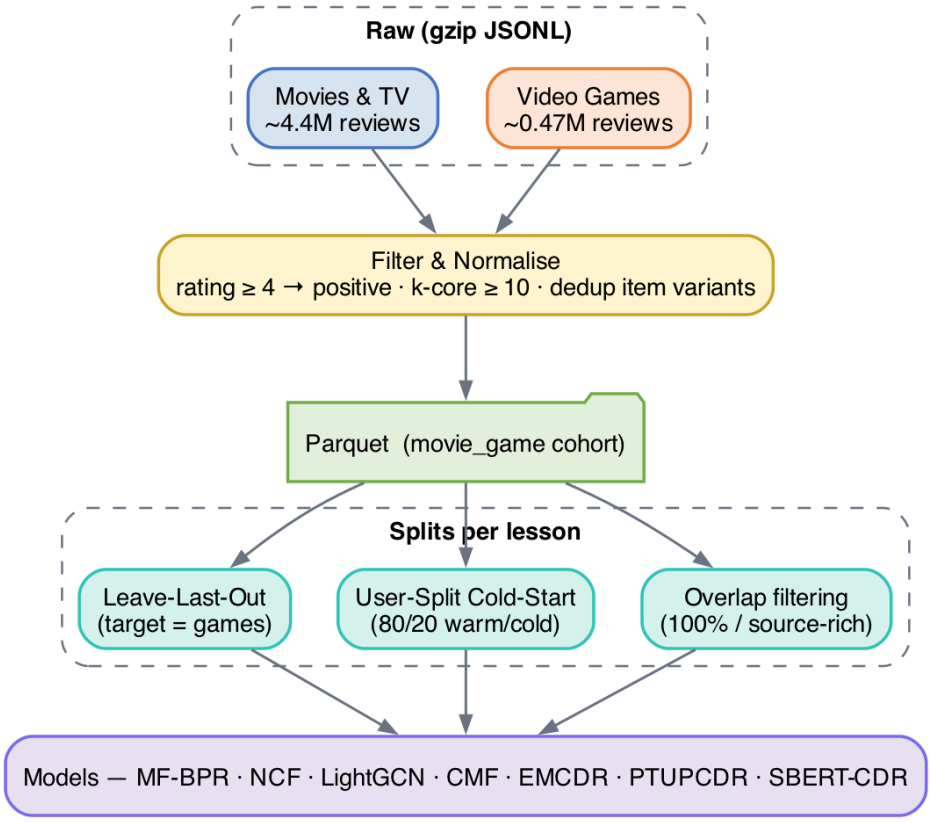


Figure 4: Data Processing Pipeline

3.4 Data Train/Test Splitting Protocol

3.4.1 Leave-last-out (LLO)

We use leave-last-out (LLO) splitting on the target domain (games): for each user, the last game interaction by timestamp becomes the test item, the second-to-last becomes the validation item, and all earlier interactions from the training set. This simulates a realistic next-item prediction scenario.

Figure 5.1: Data Splitting Visualization — Leave-Last-Out (LLO) for every user

3.4.2 User-Split cold start protocol

For Lesson 5 (cold-start), we use a cold start user-split protocol: 80% of users train with their full movie and game history, while 20% of cold start users have their game history withheld entirely. We test on cold users only.

This creates an ideal scenario in which the cross domain algorithms should have advantage over the single domain algorithms: suggesting items to users who have **never** rated a single game by relying purely on movie domain signals:

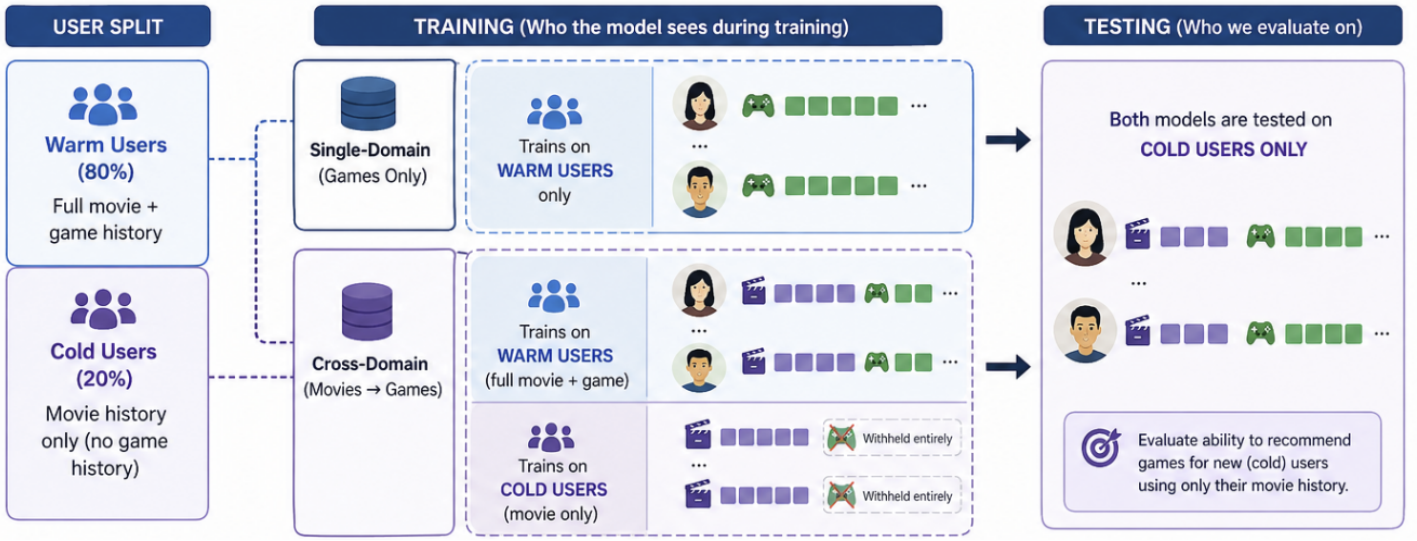


Figure 5.2: Data Splitting Visualization — Cold-Start User-Split

4. Methodology

This section explains every model we evaluate, starting from the simplest and building up to the most sophisticated. We assume no prior knowledge of recommendation systems — each model is introduced with an analogy, then its mechanism and math are explained step by step.

Meet Maya. To make the model progression intuitive, we use a running analogy: Maya runs a neighborhood rental shop that lends both movies and games. Each algorithm represents a new tool Maya adopts to recommend games, especially for users who have only rented movies. The analogies are presented in sequence to show how the methods evolve from simple single-domain recommendation to cross-domain transfer.

4.1 Evaluation Protocol

To define the quality of the recommendations, we must first establish the difficulty of the retrieval task. The evaluation protocol measures how precisely a model can identify the single correct target item within the entire catalog. We utilize a full-rank evaluation protocol that requires the model to pick the correct answer from the complete catalog of approximately >10,000 items — akin to locating one specific needle in a massive haystack. All reported Recall@10 and NDCG@10 values use full-catalog ranking rather than sampled negatives.

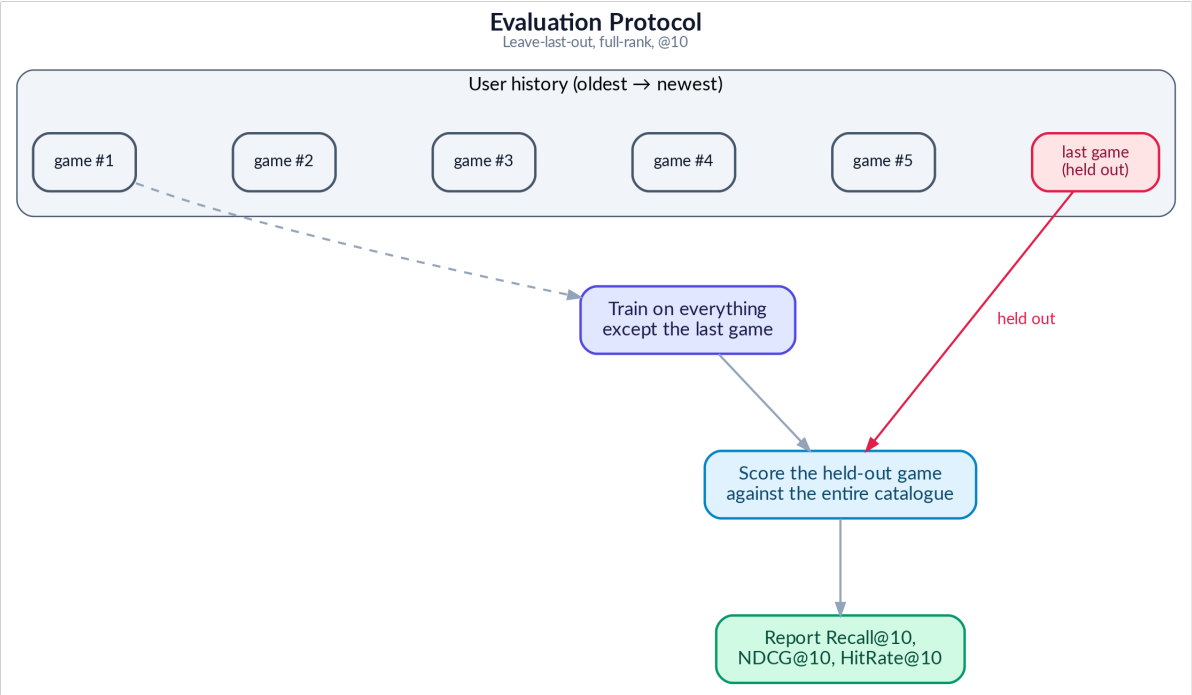
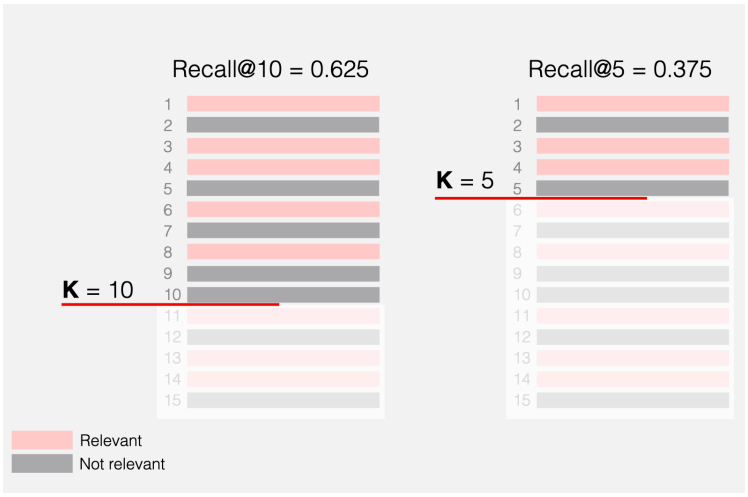


Figure 6: Evaluation Protocol

Recall@10 asks: did the correct item appear anywhere in the top 10? The ranking of the item is not important

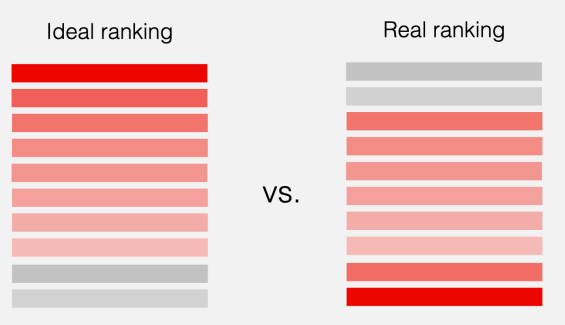
$$\text{Recall at } K = \frac{\text{Number of relevant items in } K}{\text{Total number of relevant items}}$$



NDCG@10 goes further: was it ranked first (best), third (okay), or tenth (barely made it)? Higher positions earn more credit. Its mechanism assigns a score based on the item's position, using a logarithmic

discount factor to heavily penalize lower-ranked correct items. Therefore, two models may have the same Recall@10, but the model that ranks the correct item closer to position 1 will achieve higher NDCG@10.

$$\text{NDCG@K} = \frac{DCG@K}{IDCG@K}$$



4.2 Single Domain Models

The following three models are trained using game rating data only. They have no access to movie preferences. **They only know about games** and **recommend games** based on what similar gamers have played

4.2.1 MF-BPR: Matrix Factorization with Bayesian Personalized Ranking

> **Analogy — The Notebook Method.**
>
> On her first day, Maya starts simple. For every patron and every game on her shelf she scribbles a small “taste card” — 64 numbers that try to capture things like “likes fast pace,” “prefers dark tone,” or “enjoys puzzles.” To decide whether a patron will like a game, Maya just lines up the two cards side by side and multiplies them number by number. If the totals add up high, it is probably a good match. The cards themselves are not handed to Maya — she learns them over time by watching which games each patron rented. This simple notebook already works surprisingly well, but the only way it can ever compare two cards is straight multiplication, so it misses any taste that needs a more subtle combination of traits.

MF-BPR: Architecture

Each user u and item i has a single learnable 64-dim vector, u_u, $v_i \in \mathbb{R}^{64}$. The score for a (u, i) pair is the dot product $s(u, i) = u_u \cdot v_i$. All parameters live in two matrices U ∈ ℝ^(|$\mathcal{U}$|×64) and V ∈ ℝ^(|$\mathcal{I}$|×64).

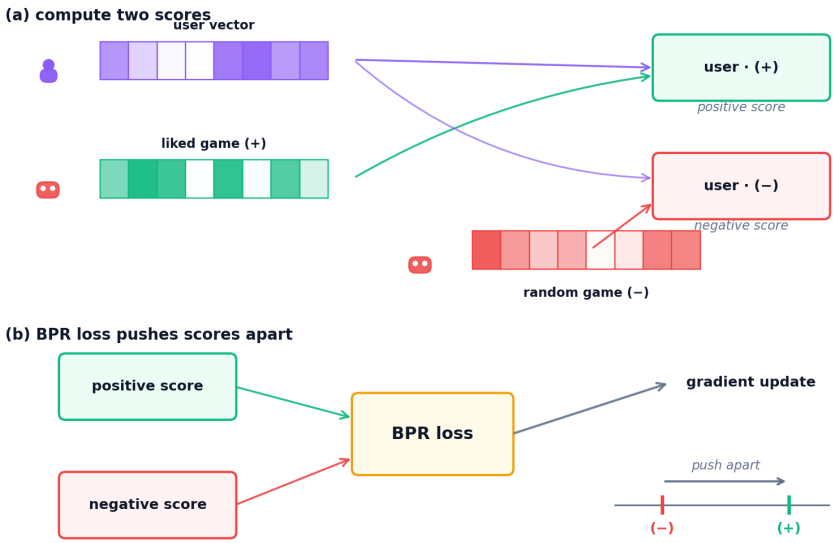


Figure 6: MF-BPR training process — user and item vectors learn from triplet sampling with BPR loss

MF-BPR: Training

We train for multiple epochs on sampled triplets (u, i^+, i^-), where i^+ is an item rated ≥ 4 by u and i^- is a uniformly sampled unobserved item. The BPR loss (Equation 1) maximizes the margin $u_u \cdot v_{i^+} - u_u \cdot v_{i^-}$ through a log-sigmoid, with L2 regularization λ.

$$\mathcal{L}_{\text{BPR}} = -\sum_{(u,\, i^+,\, i^-) \in \mathcal{D}} \ln \sigma\left(\mathbf{u}_u^\top \mathbf{v}_{i^+} - \mathbf{u}_u^\top \mathbf{v}_{i^-}\right) + \lambda \|\Theta\|_2^2$$

Equation 1: MF-BPR pairwise ranking loss.

where $\mathcal{D}$ is the set of training triplets (u, i^+, i^-); $u_u, v_i \in \mathbb{R}^{64}$ are the learnable user/item vectors; $\sigma(\cdot)$ is the logistic sigmoid; $\Theta = \{U, V\}$ collects all parameters; λ is the L2 strength

Retrieval: the trained artifacts are the user and item embedding matrices. For a user u, we score every item by dot product u·v, sort the scores, and return the top-K items.

Strengths: Simple, fast to train and a strong baseline.

Weaknesses: Every user and item is treated in isolation and a user without a learned row cannot be scored (no cold-start support).

4.2.2 NCF (NeuMF): Neural Collaborative Filtering

> **Analogy — Maya Hires an Apprentice.**
>
> After a few months, Maya notices her notebook method keeps missing oddly specific tastes — some patrons love “cozy horror,” a combination no single number on the card captures on its own. Straight multiplication just cannot express “like this trait AND that trait, but only together.” So, Maya hires a

> young apprentice (a small neural network) whose only job is to look at a patron's card and a game's card and use intuition to combine them in **non-linear** ways. She keeps her original multiplication trick as a reliable baseline and lets the apprentice add a layer of judgement on top. Now Maya catches the subtle interactions her notebook missed — but her recommendations still only know what each individual patron has personally rented.

NCF (NeuMF): Architecture

NCF fuses two parallel branches over four independent 64-dim embedding tables (GMF-user, GMF-item, MLP-user, MLP-item). The two branch outputs are concatenated and passed through a sigmoid to produce $\hat{y} \in (0, 1)$.

1. GMF branch. Element-wise product $u_u^{GMf} \odot v_i^{GMf} \rightarrow$ 64-dim. Equivalent to a matrix factorisation with per-dimension weights.

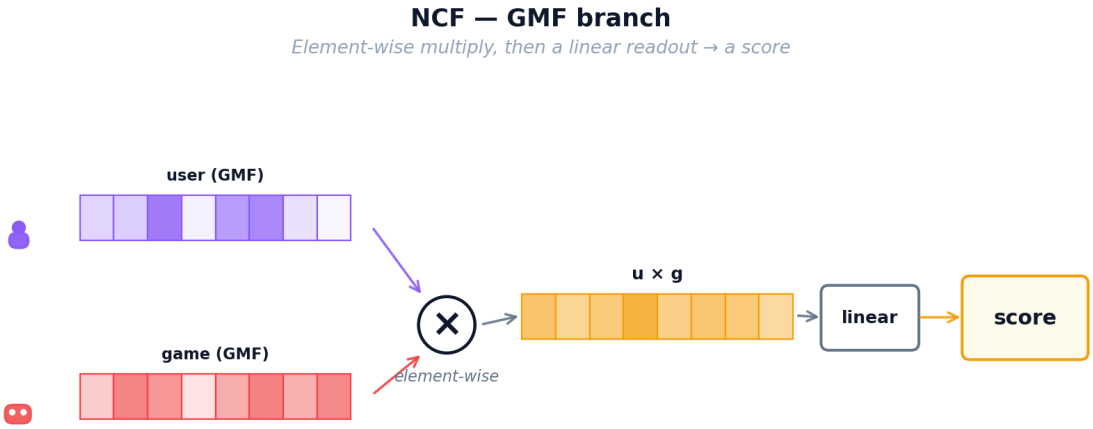


Figure 7: NCF — GMF branch: neural network layers learn non-linear user-item patterns

2. MLP branch. Concatenates u_u^{MLP} and v_i^{MLP} into a 128-dim input, then passes through ReLU layers 128 → 64 → 32. The non-linearity captures patterns a single dot product cannot.

3. Fusion. Concatenate the GMF (64-dim) and MLP (32-dim) outputs, then apply a final linear layer with sigmoid activation to produce $\hat{y} \in (0, 1)$.

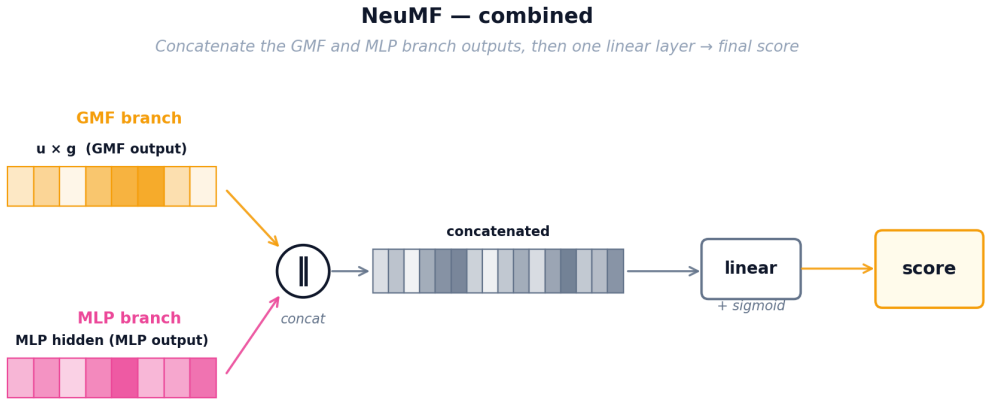


Figure 8: NCF — combined NeuMF architecture merging both branches into a final score

NCF (NeuMF): Training

NCF minimizes binary cross-entropy (Equation 2) over observed pairs (y = 1) and uniformly sampled negatives (y = 0).

$$\mathcal{L}_{\text{BCE}} = -\sum_{(u,i)\in\mathcal{D}^+\cup\mathcal{D}^-}\left[y_{u,i}\log\hat{y}_{u,i} + (1-y_{u,i})\log(1-\hat{y}_{u,i})\right]$$

Equation 2: NCF binary cross-entropy loss on observed positives and sampled negatives.

where $\mathcal{D}^+$ is the observed-interaction set (y_{u,i} = 1) and $\mathcal{D}^-$ a same-size set of uniformly sampled negatives (y_{u,i} = 0); ŷ_{u,i} ∈ (0, 1) is the NeuMF sigmoid output from the fused GMF + MLP representation. A confident wrong prediction — e.g. ŷ = 0.9 on a true negative — contributes −log(0.1) ≈ 2.3 to the loss

Retrieval: at serving time we run the learned MLP over every candidate item for user u to produce a score ŷ(u, i); sorting by ŷ gives the top-K ranking.

4.2.3 LightGCN: Graph Convolution for Recommendations

> **Analogy — The Shop Gossip Network**.
>
> Maya now realises her patrons are not isolated — they influence each other. Alice loved Zelda and also loved Witcher; Bob just loved Witcher; odds are Bob will enjoy Zelda too, even though he has never heard of it. Instead of looking only at each patron's own rental history, Maya pins up a big gossip board that connects every patron to every game they rented and every game to every patron who rented it. Taste now flows along these lines: each patron's card gets gently blended with the cards of their neighbours, and then the neighbours-of-neighbours, so the cards carry community wisdom rather than one person's history. Maya's three tools so far (notebook, apprentice, gossip board) all share one limitation though: they only know about GAMES. A patron who has only ever rented movies is still invisible to them.

LightGCN: Graph Architecture

Users and items form a bipartite graph where each interaction creates an edge. LightGCN initializes random 96-dimensional embeddings for every node, then propagates information through the graph in K=3 rounds. In each hop, a node's embedding is updated to be the weighted average of its neighbors' embeddings, with weights normalized by the square root of both nodes' degrees to prevent popular items from dominating.

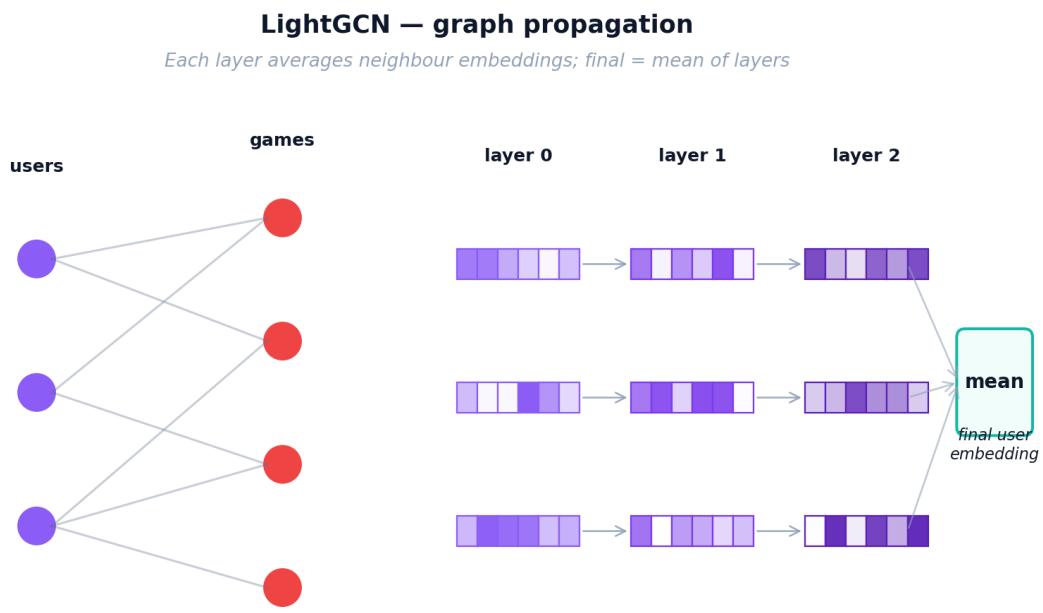


Figure 9: LightGCN graph propagation — information flows through multi-hop user-item connections

After K hops, each node has K+1 representations (from layer 0 to layer K). The final embedding is the simple mean of all layer representations. This layer-combination trick is critical: layer 0 captures the node itself, layer 1 captures direct neighbors, layer 2 captures 2-hop neighbors, and layer 3 captures 3-hop patterns. Averaging them balances local and global signals.

LightGCN: Training

LightGCN uses the same BPR loss as MF-BPR (positive items should score higher than random negatives), but the embeddings are first refined through graph propagation. In each training step: (1) sample a (user, positive, negative) triplet, (2) run K=3 layers of graph convolution to get refined embeddings, (3) compute dot-product scores, (4) compute BPR loss, (5) backpropagate gradients through all K layers back to the base embeddings. Only the base layer-0 embeddings are learnable; all propagation layers are parameter-free. This simplicity (no transformation matrices, no activation functions) is what makes LightGCN outperform more complex GCN variants.

$$\tilde{\mathbf{e}}_v = \frac{1}{K+1}\sum_{k=0}^{K}\mathbf{e}_v^{(k)} \qquad \mathcal{L}_{\text{LGCN}} = -\sum_{(u,\,i^+,\,i^-)} \ln\sigma\big(\tilde{\mathbf{e}}_u^\top\,\tilde{\mathbf{e}}_{i^+} - \tilde{\mathbf{e}}_u^\top\,\tilde{\mathbf{e}}_{i^-}\big) + \lambda\,\|\mathbf{E}^{(0)}\|_2^2$$

Equation 3: LightGCN layer-combined embeddings and BPR loss.

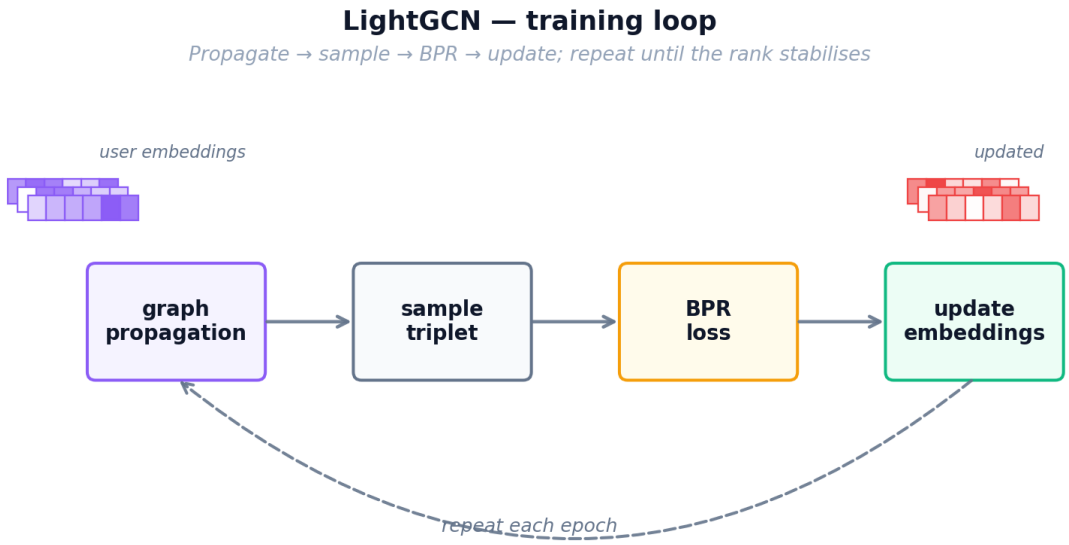


Figure 10: LightGCN training loop — graph propagation followed by BPR loss optimization

Retrieval: after propagation, the final user and item embeddings are dotted to score every item for a given user, and the top-K scores are returned.

> **Key takeaway — why LightGCN is the default champion.**
>
> LightGCN propagates user–item embeddings through the bipartite interaction graph; neighborhood structure substitutes for the expressive transformation NCF adds explicitly. Across Lessons 2–5 it is the consistent leader on standard leave-last-out — 1.7×–5.8× the best CDR model on a mixed subgroup, still leading even at 100% overlap. Its weakness is the cold-start case (Lesson 6): without target-domain interactions a graph has nothing to propagate, and LightGCN collapses to near-zero.

4.3 Cross Domain Models

The following three models are trained using both movie and game data. They attempt to transfer knowledge from the movie domain to the game domain. Think of them as translators: they try to convert what they know about a user's movie taste into game recommendations. This is especially valuable for "cold-start" users who have watched movies but never played games.

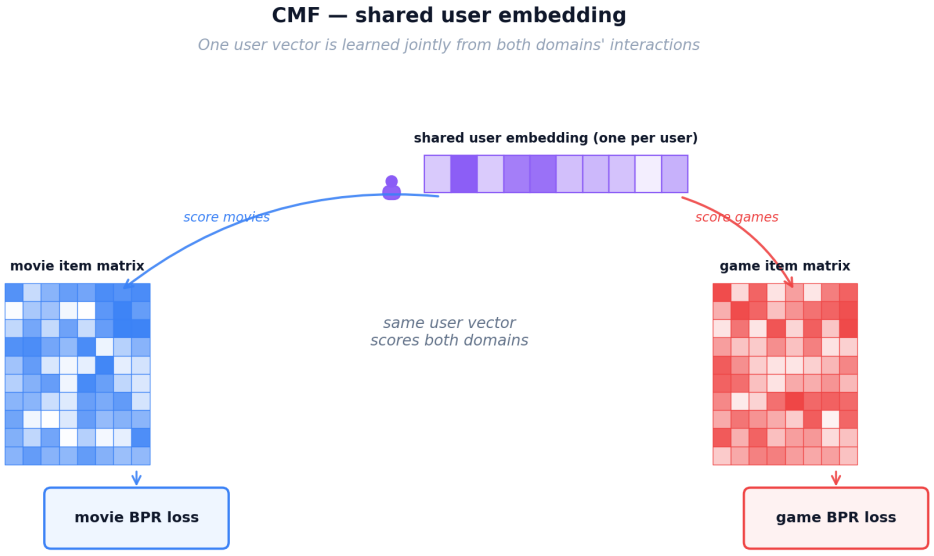
4.3.1 CMF: Collective Matrix Factorization

> **Analogy — One Shared Customer Card.**
>
> Maya now faces her real problem: she wants to recommend games to movie-only patrons, but her movie ledger and her game ledger were always kept as two separate notebooks that never talked to each other. Her first attempt to bridge them is the simplest thing possible: throw away the two patron notebooks and keep ONE single taste card per patron that must work for both sides of the shop. Whenever Maya learns something from a movie rental, she updates the same card she uses for games, and vice versa. This forces the card to capture traits that genuinely transfer — an "action lover" card now helps with both Die Hard and Doom. The downside is that cramming two very different worlds into one card waters each of them down: the shared profile can never specialize as deeply as a game-only or movie-only card could.

CMF Architecture

Shared user matrix. CMF keeps one user matrix $U \in \mathbb{R}^{(|\mathcal{U}| \times 96)}$ shared by both domains, plus domain-specific item matrices $V^M \in \mathbb{R}^{(|\mathcal{I}^M| \times 96)}$ for movies and $V^G \in \mathbb{R}^{(|\mathcal{I}^G| \times 96)}$ for games. Every user has exactly one 96-dim vector used in both domains.

Item matrices stay private. V^M receives only movie gradients, V^G only game gradients; the shared U is therefore the sole transfer channel between the two domains.



CMF Training

Sampling. Each step draws one BPR triplet per domain: (u, i^{M+}, i^{M-}) and (u, i^{G+}, i^{G-}) with uniform negatives. Users present in only one domain still contribute — their row of U is pulled by that domain's triplets alone.

Joint objective. The total loss is a convex combination of the two per-domain BPR terms sharing U (Equation 4). $\alpha \in [0, 1]$ trades movie signal ($\alpha \to 1$) for game signal ($\alpha \to 0$); our default is $\alpha = 0.5$.

$$\mathcal{L}_{\text{CMF}} = \alpha\,\mathcal{L}_{\text{BPR}}^{\text{movie}}(\mathbf{U}, \mathbf{V}_m) + (1-\alpha)\,\mathcal{L}_{\text{BPR}}^{\text{game}}(\mathbf{U}, \mathbf{V}_g) + \lambda\,\|\Theta\|_2^2$$

Equation 4: CMF joint two-domain loss with shared user matrix U.

where U is the shared user matrix; V^M, V^G are the domain-specific item matrices; L_BPR^M, L_BPR^G are the per-domain BPR losses (same form as Equation 1); $\alpha \in [0, 1]$ balances the two domains; $\Theta = \{U, V^M, V^G\}$; λ is the L2 strength.

Retrieval: the shared user embedding is dotted with the target-domain item matrix V^G to score every game, then sorted to produce the top-K ranking.

CMF Limitations

The shared u_u is a compromise vector that must serve both domains and is therefore mediocre at each. In our experiments CMF produces the weakest CDR transfer — especially at cold-start, where u_u has been shaped almost entirely by movie data (≈95% of interactions).

4.3.2 EMCDR: Cross-Domain Embedding Mapping

> **Analogy — Maya Hires a Translator.**
>
> CMF taught Maya that one shared card sacrifices too much specialization. Her next idea is better: keep the two specialized notebooks (one perfectly tuned for movies, one perfectly tuned for games), then hire a translator who converts between them. For every patron who has rented in BOTH domains — the overlap customers — Maya can see the same person's movie card AND their game card, so she can teach the translator: "when the movie card looks like this, the matching game card looks like that." Once trained, the translator lets Maya take a movie-only patron's card and translate it straight into a game-side card, so she can recommend games as if they had always been a game patron. This is a big leap over CMF because neither specialized card had to compromise. The weak spot is that Maya only hires one translator, and that one translator must use the same rules for every single patron.

EMCDR Architecture

Two independent spaces. EMCDR trains one MF-BPR per domain: a source-domain model produces user and item embeddings for movies (U^M, V^M), and a second, separate target-domain model produces its own embeddings for games (U^G, V^G). The same person is represented by two unrelated vectors — one in movie space, one in game space — because each domain is modelled in isolation ($u_u^M \neq u_u^G$).

A learned translator. A small multilayer perceptron (MLP) learns to map a user's movie-space vector to its game-space counterpart: $f(u_u^M) \approx u_u^G$. Once trained, any user with a movie-side vector — including cold-

start users who have never rated a game — can be placed into game space and scored against every item in V^G.

Supervised by overlap. The translator is trained only on the overlap users $\mathcal{U}_o$ = {u : u has interactions in both domains}. These users provide paired (movie-vector, game-vector) examples the MLP can learn from — without any overlap, the translator has no supervision signal.

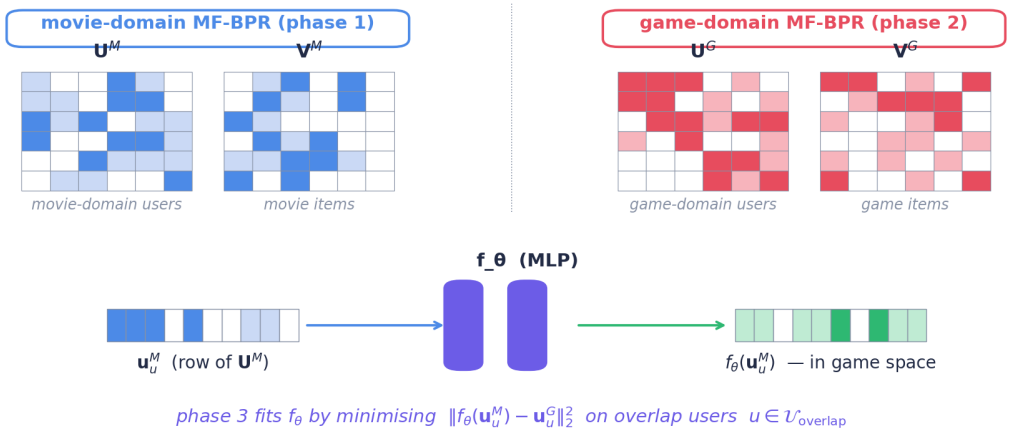


Figure 12: EMCDR architecture — two independent MF-BPR spaces bridged by the learned translator fθ.

EMCDR Training

Training proceeds in three sequential phases:

Phase 1 — movie MF-BPR. Train the source-domain MF-BPR on movie interactions until convergence to obtain $\boldsymbol{U}^M$ and $\boldsymbol{V}^M$ (user and item embedding tables in movie space).

Phase 2 — game MF-BPR. Independently train the target-domain MF-BPR on game interactions to obtain $\boldsymbol{U}^G$ and $\boldsymbol{V}^G$. The two models share no parameters — each domain is learned in isolation so it can optimise for its own local structure.

Phase 3 — fit the translator. Freeze both MF-BPR models. Using the overlap users only, fit f_θ to regress $\boldsymbol{u}_u^G$ from $\boldsymbol{u}_u^M$ with the mean-squared-error loss shown in Equation 5. Any gradient-based optimiser works; train until the loss on a held-out subset of overlap users stops improving.

$$\mathcal{L}_{\text{EMCDR}} = \sum_{u \in \mathcal{U}_{\text{overlap}}} \left\| f_\theta\left(\mathbf{u}_u^{\text{movie}}\right) - \mathbf{u}_u^{\text{game}} \right\|_2^2$$

Equation 5: EMCDR phase-3 mapping loss (MSE on overlap users).

where $\mathcal{U}_o$ is the set of overlap users; u_u^M, u_u^G are the user embeddings learned independently in Phases 1 and 2; f_θ is the MLP translator fit in Phase 3 by regressing the game-side vector from the movie-side vector.

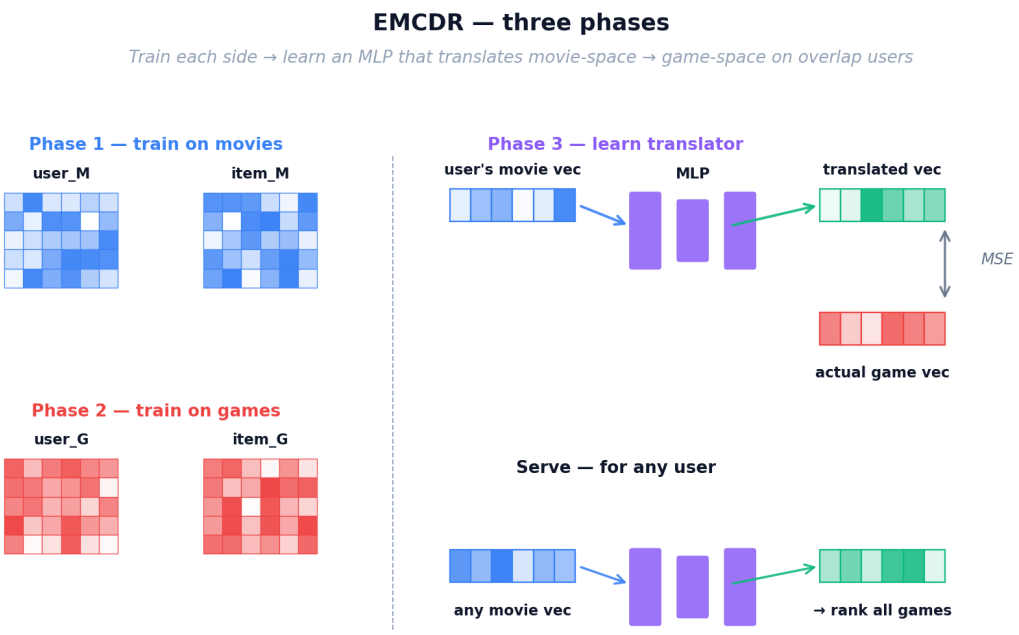


Figure 13: EMCDR three-phase training — separate domain models, then a learned mapping network.

Retrieval: at serving time a user's movie-side vector is passed once through the trained translator to land in game space, then dotted with every game embedding in V^G; sorting the scores gives the top K ranking. Because the target-side user matrix is never touched, users with zero game history are natively supported.

4.3.3 PTUPCDR: Personalized Transfer with Experts

> **Analogy — A Team of Specialist Translators.**
>
> Maya realizes EMCDR uses one global translator to map every movie fan into game space. But a sci-fi lover and a rom-com fan don't translate the same way — averaging them loses information. PTUPCDR fixes this by hiring a small team of specialist translators plus a receptionist (the gate network). For every user, the gate reads their movie vector and decides which specialists to trust and by how much. A sci-fi fan is routed mostly through the sci-fi specialist, a comedy fan mostly through the comedy specialist. Every user now gets their own personalized translation rule, not one-size-fits-all.

PTUPCDR keeps EMCDR's three-phase recipe but upgrades Phase 3: the single MLP translator f_θ is replaced by a mixture of K expert MLPs, steered by a gating network that produces per-user expert weights. Everything else — the source and target MF-BPR models, the use of overlap users as supervision, the serving pipeline — is identical to EMCDR. Only the shape of the translator changes.

PTUPCDR Architecture

The translator is no longer a single function. At inference time the pipeline has five steps:

Step	Action	Formula / Description
1	**Input Embedding**	The user's movie-side embedding $\mathbf{u}_u^M \in \mathbb{R}^D$ is obtained from the pre-trained source-domain MF-BPR model.
2	**Gating Network**	The gate network g computes K non-negative weights that sum to 1: $\mathbf{g}(\mathbf{u}_u^M) = \text{softmax}(\mathbf{W}_g \cdot \mathbf{u}_u^M)$.
3	**Expert Mapping**	Each expert $k \in \{1, ..., K\}$ is an independent MLP that maps the source embedding to the target game space: $\mathbf{f}_k(\mathbf{u}_u^M)$.
4	**Personalized Translation**	The final translated vector $\mathbf{F}(\mathbf{u}_u^M)$ is the weighted sum of all expert outputs: $\mathbf{F}(\mathbf{u}_u^M) = \Sigma_{k=1}^K g_k(\mathbf{u}_u^M) \cdot \mathbf{f}_k(\mathbf{u}_u^M)$.
5	**Scoring & Ranking**	The final score for game g is the dot product between the personalized translation and the game embedding $\mathbf{v}_g^G$: $s(u, g) = \mathbf{F}(\mathbf{u}_u^M) \cdot \mathbf{v}_g^G$. The top-K items are returned.

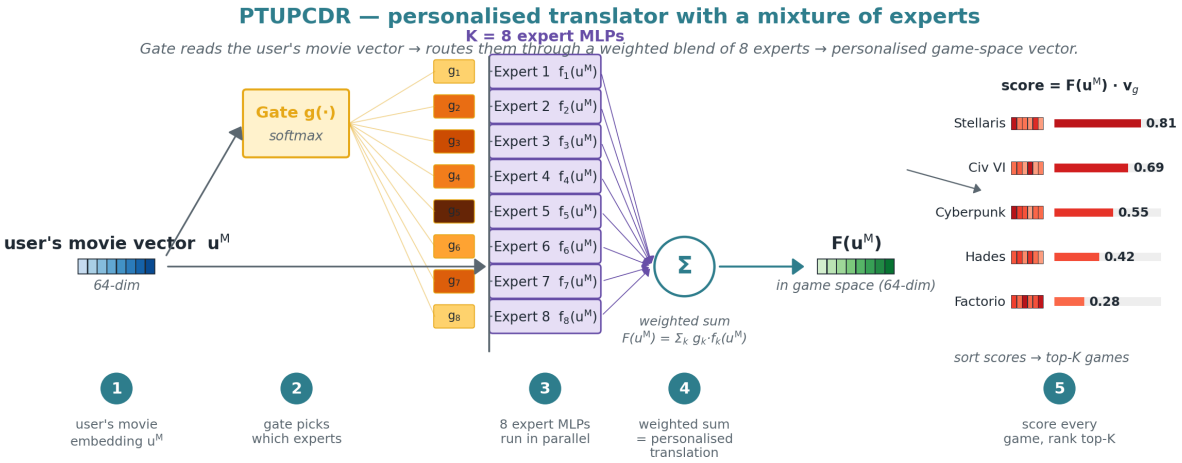


Figure 14: PTUPCDR architecture.

PTUPCDR Training

Phases 1 and 2 are identical to EMCDR: train MF-BPR independently on movies to get U^M, V^M, then a second MF-BPR on games to get U^G, V^G. **Phase 3 — fit the experts and gate.** Freeze both MF-BPR models. On the overlap users only, jointly train the gate's weights W_g and all K experts {f_k} to minimise the mean-squared error between the predicted translation $F(u_u^M)$ and the user's actual game-side vector u_u^G (Equation 6). Any gradient-based optimizer with early stopping on a held-out overlap-user validation split works.

$$\mathcal{L}_{\text{PTUPCDR}} = \sum_{u \in \mathcal{U}_{\text{overlap}}} \left|\left| \sum_{k=1}^{K} g_k\left(\mathbf{u}_u^{\text{movie}}\right) f_k\left(\mathbf{u}_u^{\text{movie}}\right) - \mathbf{u}_u^{\text{game}} \right|\right|_2^2$$

Equation 6: PTUPCDR mixture-of-experts personalised mapping loss.

where $\mathcal{U}_o$ is the set of overlap users, f_k is the k-th expert MLP, and g_k = softmax(W_g · u_u^M)_k is the k-th entry of the softmax gate. Each user's movie-side vector is routed through its own expert mix — a user-specific translation rule that a single global f_θ (EMCDR) cannot produce.

Retrieval: at serving time the gate is applied to the user's movie-side vector to produce K weights; each expert is run on the same vector; the weighted sum is the personalised game-space translation, which is dotted with every game embedding to score and rank the top-K items. Users with zero game history are natively supported because the target-side user matrix is never needed.

4.4 Content-Based and Post Processing Models

The remaining two approaches don't rely on collaborative filtering. SBERT matches items by their text descriptions, while co-occurrence reranking boosts scores based on statistical patterns in the data. Both are training-free and model-agnostic.

4.4.1 SBERT-CDR: Content-Based Semantic Matching

> **Analogy — Reading the Back of the Box.**
>
> Collaborative models require rental history, which fails for brand-new games or patrons. To solve this "cold-start" problem, Maya reads the back of the box—titles and summaries—using a pre-trained language model as reading glasses. This turns text into semantic cards based on meaning rather than ratings. Now, "sci-fi action with robots" on a movie aligns with "sci-fi thriller featuring androids" on a game, even without previous rentals. This allows Maya to recommend new or niche items that collaborative models may struggle with, especially when interaction history is sparse.

SBERT-CDR Architecture

SBERT-CDR uses a pre-trained Sentence-BERT model (all-MiniLM-L6-v2) to encode each item's text into a dense 384-dimensional vector. The input text is formatted as "title: [title]. description: [desc]" and passed through the transformer's 6 attention layers to produce a single fixed-size vector per item.

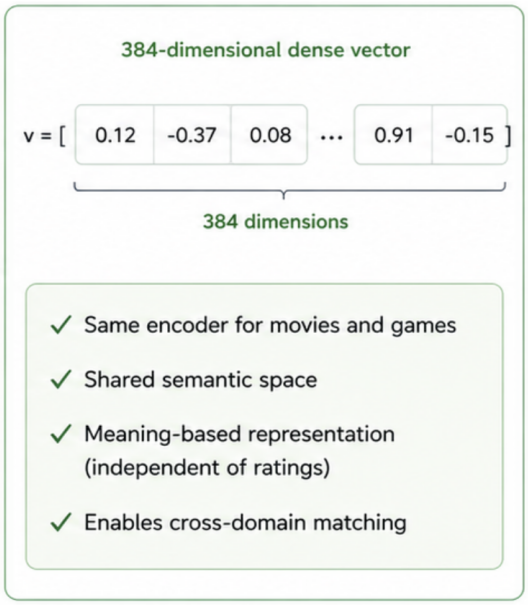


Figure 16: SBERT encoding — item text is transformed into a 384-dim semantic vector

The key insight is that movies and games are encoded into the same 384-dimensional space using the same encoder. Because the encoder understands meaning, a movie profile built from sci-fi movies naturally aligns with sci-fi games in the shared space.

SBERT-CDR Inference & Scoring

At inference, a user profile $\mathbf{p}_u$ is built by averaging the embeddings of all items the user has interacted with.

$$\mathbf{p}_u = (1 / |\mathcal{I}_u|) \Sigma_{i \in \mathcal{I}_u} \mathbf{v}_i$$

Equation 7: User Profile construction via mean pooling.

Where:

- $\mathbf{p}_u$ is the computed 384-dimensional user profile vector for user *u*.
- $|\mathcal{I}_u|$ is the total number of items the user *u* has interacted with (movies and games).
- $\Sigma_{i \in \mathcal{I}_u}$ is the summation over all item embeddings $\mathbf{v}_i$ for items *i* belonging to the set of interacted items $\mathcal{I}_u$.

- $\mathbf{v}_i$ is the 384-dimensional semantic embedding vector for item *i* generated by the Sentence-BERT encoder.

The final score for a game *g* is the cosine similarity between the user profile and the game embedding $\mathbf{v}_g$:

$$\text{score}(u, g) = (\mathbf{p}_u \cdot \mathbf{v}_g) / (||\mathbf{p}_u|| \, ||\mathbf{v}_g||)$$

Equation 8: Scoring function via Cosine Similarity.

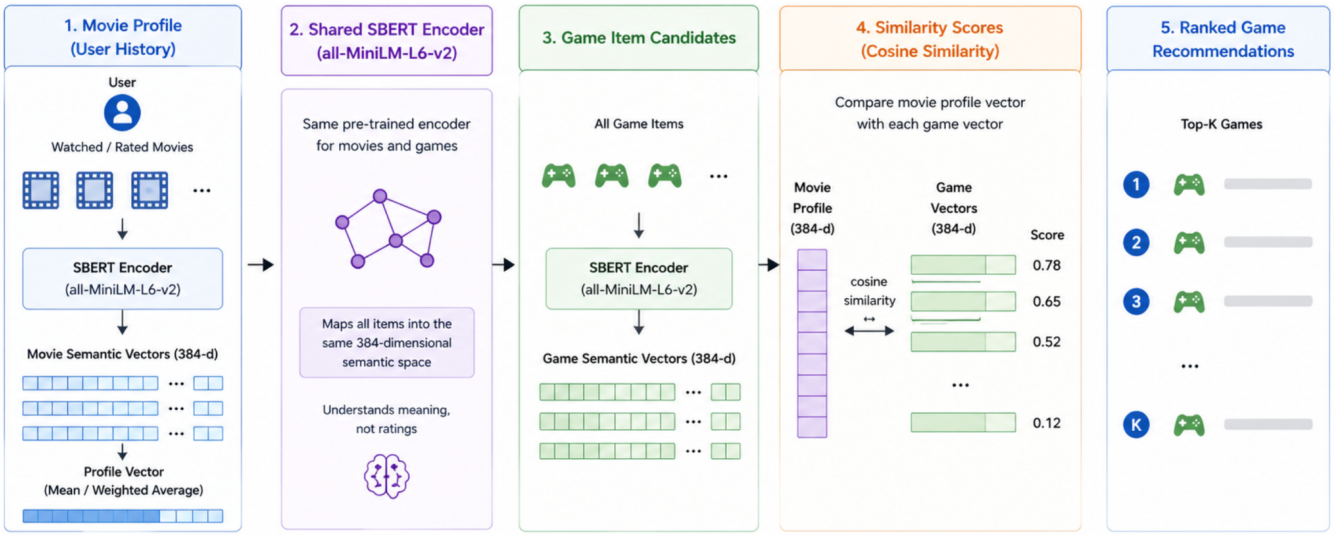


Figure 17: SBERT-CDR cross-domain flow — movie profiles match game items through shared semantic space

5. Experiments and Results

5.1 Lesson 1: Explicit vs Implicit Ranking

At first, as Amazon Data Ratings consist of exclusively explicit ratings(1-5 starts), so naturally we did our initial benchmarks with only the algorithms that are good at predicting user ratings. But, we were baffled by the fact that the initial metrics were very disappointing. This raised a question whether our methods are valid. From then we learnt that predicting a 1–5 rating is fundamentally different from ranking items a user picked over ones they skipped.

This lesson shows how choosing the wrong loss function can silently break every downstream experiment.

> **HYPOTHESIS**
>
> *Will the implicit ranking objective (BPR) outperform the explicit point-predicting approach (SVD) on Recall@10?*

Setup

We train the same matrix-factorization architecture twice on identical data — **once minimizing RMSE regression loss, once minimizing BPR pairwise ranking loss** — and compare metrics.

- **Game Domain only – MF-Explicit (SVD)** — with MSE loss
- **Game Domain only – MF-Implicit BPR** — with BPR ranking loss

Evaluation: leave-last-out on games, Recall@10, NDCG@10.

Dataset

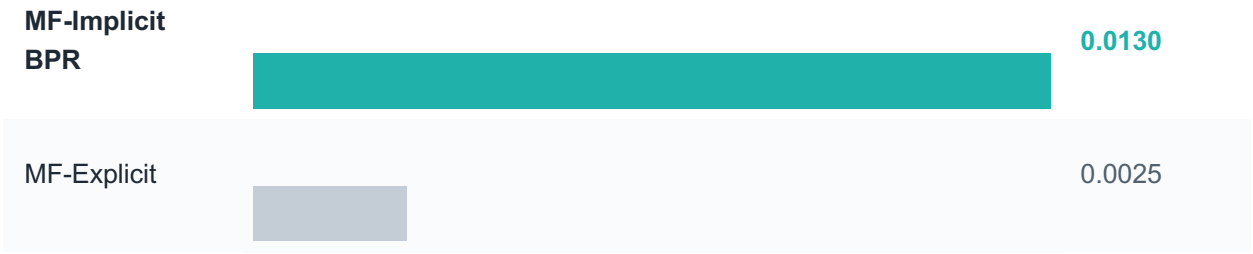
Property	Value
Users	1,000,000 (randomly sampled from original unfiltered 3.7M users)
Game Ratings	337K (implicit rating ≥ 4, ratings from the sampled 1M users, originally unfiltered 1.4M ratings)
Game items	11,706
Item k-core	games ≥ 10
Split	Leave-last-out on games

Results

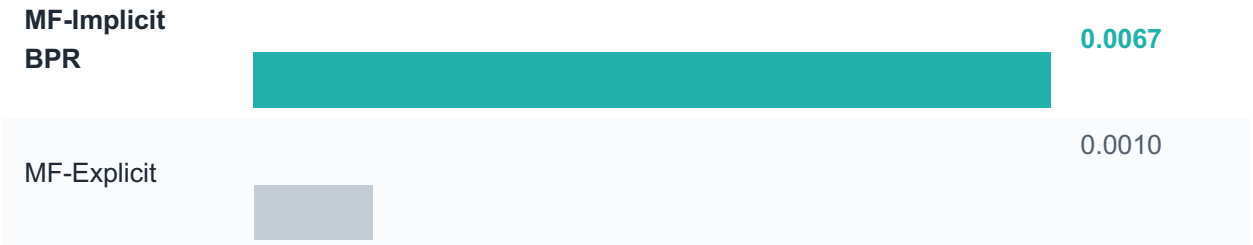
With **MF- Implicit BPR** outperforms **MF-Explicit**, the **Implicit** variant extracts **5.2×** more signal than **MF-Explicit** on Recall@10.

Model	Loss Function	Recall@10	NDCG@10
MF-Implicit BPR	BPR on ranking	**0.0130**	**0.0067**
MF-Explicit	RMSE on star predicting	0.0025	0.0010

Recall@10 **5.2× win for BPR**



NDCG@10 **6.7× win for BPR**



OBSERVATION

Confirmed — BPR wins 5.2× on Recall@10. The ranking loss directly optimizes the quantity that evaluation metrics measure, while RMSE optimizes prediction accuracy on ratings that may not correlate with top-K relevance.

Summary

Lesson 1 asks whether we should train on explicit 1–5 ratings or treat ratings implicitly as positive signals.

The takeaway for the rest of the report: ranking losses are the right method for recommendation, and every subsequent experiment should be trained with implicit ratings.

Finding	Evidence
BPR crushes MF-Explicit	Recall@10 **0.0130** vs 0.0025 — **5.2× win**; NDCG **6.7×**

Reflection

This lesson reproduces one of the most common results in collaborative filtering: for top-K ranking, the training objective is much more important than the architecture itself. Using RMSE is simply the wrong way to evaluate if we showed the user something they would actually click.

This is a basic mistake we should not make. But we are glad we learned this lesson so early on: **always pick a loss that matches the metric you will actually be evaluated on.** The very same model can be trained with multiple loss functions to choose from, and now we know exactly the reasons behind that!

> **NEXT UP**
>
> **Lesson 1** settled the training-objective question: **implicit ranking** beats explicit rating prediction. Next lesson we will benchmark both Single Domain Recommendation algorithms and Cross Domain Recommendation algorithm**:** Will CDR outperform SDR?

5.2 Lesson 2: Low Overlap Severely Limits CDR

When we read about Cross-domain recommenders (CDRs), we always see complicated explanations of how they work, and then at the end, metrics showing how CDRs scores successfully beat SDR counterparts. But is this always true? Or is there a catch? So long as we have two related domains and some overlapping users, should we always expect CDRs to outperform?

This lesson tests CDR at a realistic ~5% overlap — the ratio you would see on raw Amazon Reviews data without any filtering. We wanted to see if the *magic* of CDR still works when bridge users are very few.

> **HYPOTHESIS**
>
> *Can collaborative CDR models (CMF, EMCDR, PTUPCDR) beat a single-domain graph recommender (LightGCN) when only 5.2% of users appear in both domains?*

Setup

We expand the portfolio from two MF models to six — three single-domain models **(MF-BPR, NCF, LightGCN)** against three CDR models **(CMF, EMCDR, PTUPCDR)** — all trained on the same mixed 1M-user sampled group where only 5.2% of users appear in both domains.

- **Single-domain** — MF-BPR (pairwise MF), NCF (NeuMF), LightGCN (graph) – only trains and tests on Game domain dataset.
- **CDR** — CMF (joint MF), EMCDR (mapping), PTUPCDR (personalized mapping) – trains and tests on Movie-Game cross domain dataset.

Evaluation: leave-last-out on games, Recall@10 (same 1M-user subgroup as Lesson 1).

Dataset

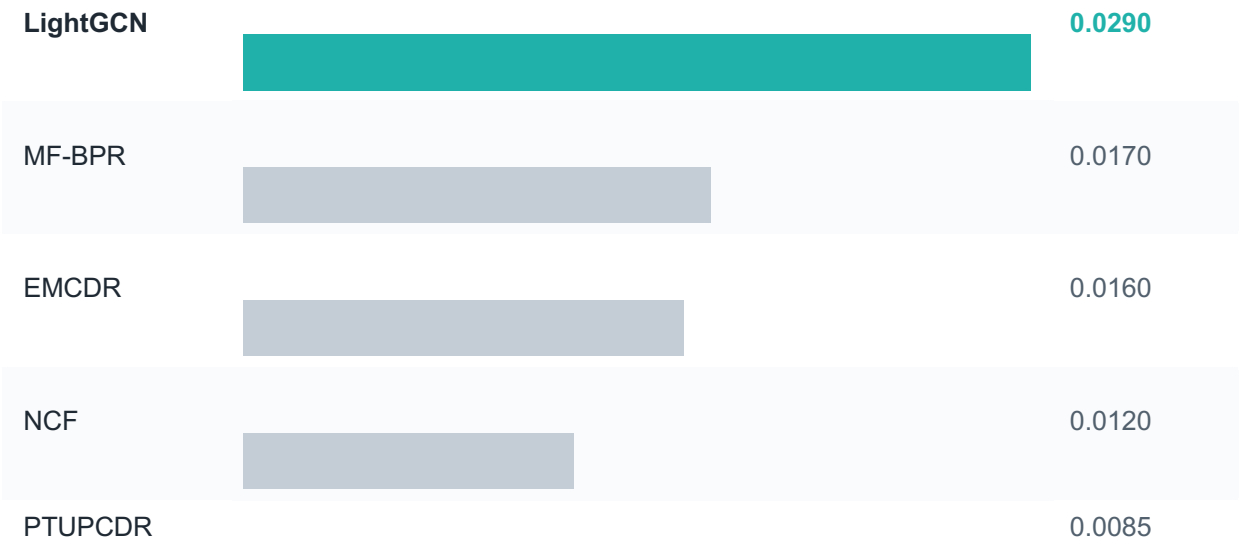
Property	Value
Users	1,000,000 (no user k-core, random sampled from original 3.7M users)
Overlap	52,281 (5.2%) — same ratio as in original dataset
Movie - game ratings	Movie ratings 1.80M - Game ratings 337K (rating ≥ 4)
Movie - game items	Movie items 45,933 - Game items 11,706
Item k-core	movies ≥ 20, games ≥ 10
Split	Leave-last-out on games

Results

At 5.2% natural overlap, **LightGCN leads by 1.8× over the best CDR model (EMCDR)** — all three CDR models trail even the simplest single-domain baseline.

Model	Family	Recall@10	NDCG@10	% gap
LightGCN	Graph (single-domain)	**0.0290**	**0.0157**	0%
MF-BPR	MF (single-domain)	0.0170	0.0093	−41.4%
EMCDR	Mapping (CDR)	0.0160	0.0082	−44.8%
NCF	Neural (single-domain)	0.0120	0.0069	−58.6%
PTUPCDR	Personalized mapping (CDR)	0.0085	0.0054	−70.7%
CMF	Joint MF (CDR)	0.0050	0.0022	−82.8%

Recall@10 **LightGCN leads · CDR trails**



CMF		0.0050

> **KEY FINDING**
>
> **Confirmed —** LightGCN wins 1.7× over MF-BPR and 1.8× over EMCDR (best CDR). At 5.2% overlap, cross-domain mappings have too few shared users to learn meaningful transfer — and get beaten even by baseline MF.

Summary

Lesson 2 asks whether, at a realistic low-overlap ratio, any CDR model can beat the best single-domain approach. **All three CDR models underperform the simplest single-domain baseline.**

Finding	Evidence
LightGCN wins	Recall@10 **0.0290** — **1.7× over MF-BPR, 1.8× over EMCDR** (best CDR)
CDR underperforms single-domain	All three CDR models lose to MF-BPR, the simplest SDR
MF-BPR holds up	Simplest model beats all CDR on full-rank — pairwise ranking signal is strong even on sparse data

Reflection

When we first started the benchmark between CDR and SDR, we had very high hopes for the Cross-Domain Recommenders, strongly believing that their complex, advanced learning algorithms would easily outperform the traditional single-domain approaches. The actual results, which showed SDR clearly winning, were genuinely disappointing and forced us to question our assumptions.

We initially thought the poor performance was due to settings that weren't quite right. We ran many tests by tweaking the hyperparameters for EMCDR, but no matter how much we tuned the parameters, the model consistently performed worse than the SDR approaches. At one point, we even thought about abandoning the project idea.

This forced us to look back at the CDR articles and consult with ChatGPT. We noticed that existing research often uses datasets where a high number of users overlap (e.g., Amazon books → movies with 50%+ overlap). This observation provided the critical insight: when these same models are applied to real, raw data— where user overlap is under 10%—CDR simply fails, which is exactly what Lesson 2 demonstrated.

This finding highlights a common pattern: **methods that work beautifully on perfect test datasets might fail on the real-world data.** Running a simple baseline on your real data before trusting any published benchmark is a crucial safety step. The true value of Lesson 2 was not the bad result itself, but the important lesson it taught us about data requirements.

> **NEXT UP**
>
> **Lesson 2 leaves an ambiguity: is CDR fundamentally weak, or just starved of bridge users at 5% overlap?** The only way to know is to filter the users with ratings on both domains – 100% overlap and re-run the same six models.

5.3 Lesson 3: Overlap Filtering Rescues CDR

Lesson 2 was a disaster for CDR. The models we had so much hope for failed against simple baselines because the user overlap was too low (5.2%). This left us with a huge, uncomfortable question: Is CDR fundamentally flawed? This lesson is the answer. We filter out the low-overlap users and re-run the benchmark on a 100% overlap subgroup.

> **HYPOTHESIS**
>
> *When every user has history in both domains (100% overlap), do CDR models finally become competitive with LightGCN?*

Setup

Same six models as Lesson 2. What changes is the user group: users with **≥ 10 total interactions AND ≥ 5 movies and ≥ 1 game** — 19,880 bridge users, 100% overlap by construction.

- **Single-domain** — MF-BPR, NCF, LightGCN
- **Cross-domain** — CMF, EMCDR, PTUPCDR

Evaluation: leave-last-out on games, full-rank Recall@10 (same protocol as L1–L2).

Dataset

Property	Value (changes from Lesson 2)
Users	**19,880** (was 1,000,000 sampled)
Overlap	**19,880 (100%)** — was 5.2%
Movie · game ratings	430,736 · 87,613
Movie · game items	40,288 · 10,034
User k-core	≥ 10 total ratings (new) ≥ 5 movies ratings (new) ≥ 1 games ratings (new)
Split	Leave-last-out on games

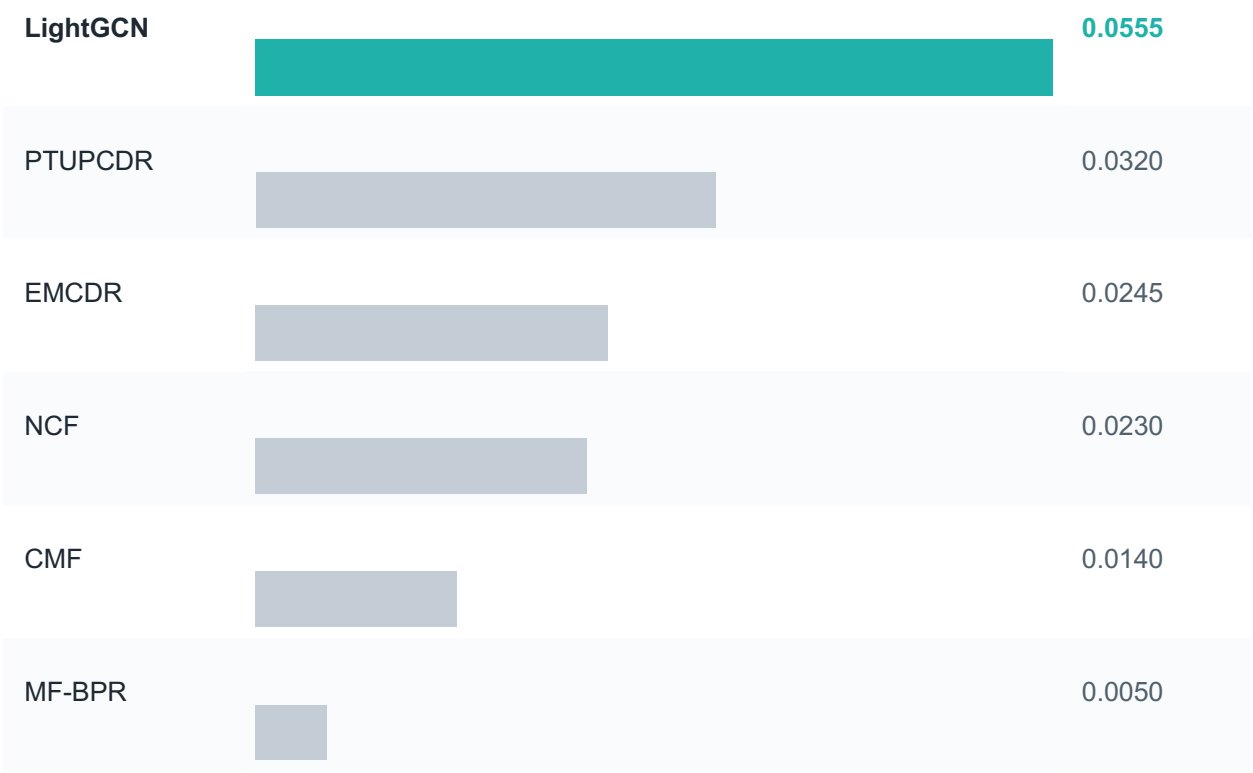
Results

Interesting! Every CDR model jumps dramatically. **PTUPCDR surges +276% , while CMF +180%, EMCDR +53%**. LightGCN still leads but CDR is now a serious competitor.

Model	Family	Recall@10	NDCG@10	% gap
LightGCN	Graph (single-domain)	**0.0555**	**0.0311**	—
PTUPCDR	Personalized mapping (CDR)	0.0320	0.0179	−42.3%
EMCDR	Mapping (CDR)	0.0245	0.0115	−55.9%
NCF	Neural (single-domain)	0.0230	0.0130	−58.6%
CMF	Joint MF (CDR)	0.0140	0.0073	−74.8%

MF-BPR	MF (single-domain)	0.0050	0.0030	−91.0%

Recall@10 **LightGCN leads · CDR closes in**



KEY FINDING

Confirmed — 100%-overlap filtering rescues CDR. PTUPCDR closes the gap from −71% (L2) to −42% (L3); every CDR model improves substantially. Overlap is the single most decisive variable for cross-domain transfer.

Summary

Lesson 3 asks whether the poor L2 CDR results were caused by structural weakness or simply too few bridge users.

The takeaway: **CDR is not fundamentally weak** — it was starved of bridge(overlapping) users in Lesson 2. With enough bridged signal, CDRs now strongly compete with SDRs because they have enough data signal for them to work on.

Finding	Evidence
Overlap filtering rescues CDR	PTUPCDR 0.0085 → 0.0320 (+276%); CMF +180%; EMCDR +53%
LightGCN still leads	Recall@10 = 0.0555 — up from 0.0290 in L2 (+91%). Graph structure scales with denser per-user game data.

Reflection

After the huge disappointment of Lesson 2, where we started to doubt the entire idea of CDR, the results from Lesson 3 were a massive relief.

It showed the complex CDR models we chose were not flawed—they were just starved of the necessary bridge data. Generally, this lesson is critical: when a method underperforms, is it we are using the **wrong** tool for the job? Or it's the **right** tool but with **not** enough data. We confirmed that the core problem wasn't the CDR algorithm itself, but the data setup.

> **NEXT UP**
>
> Lesson 3 confirmed overlap is decisive, but the 100%-overlap subgroup is still a mix of user types — some movie-heavy, some game-heavy**.** But LightGCN still leads. What would it take for CDR to outpace the LightGCN?

5.4 Lesson 4: Source-Rich / Target-Sparse

Lesson 3 gave us a big relief: Cross-Domain Recommendation is not broken; it just needs enough overlapping users. But even with 100% overlap, the LightGCN model still won. LightGCN is very strong SDR.

So, the quest is still on. We need to find the specific dataset—the one setup where CDR can outperform. ChatGPT mentions this might happen when a user has a *very* rich source domain (movies) but only *very* little target domain history (games). This is the Source-Rich / Target-Sparse; sweet spot that CDR was designed for.

This lesson is our next move: we tighten the data filter even more, demanding users have even **richer** movie history.

> **HYPOTHESIS**
>
> *Does CDR close the gap to LightGCN further when users have richer movie history but proportionally less game history?*

Setup

Same six models, same 100%-overlap subgroup structure as Lesson 3. Only the movie-history floor changes: user must have at least **10 movies ratings (was ≥ 5)**. This removes ~5.5K movie-poor users and drops game interactions by 33% — penalizing game-signal-heavy models and rewarding models that leverage movie history.

- **Single-domain** — MF-BPR, NCF, LightGCN
- **CDR** — CMF, EMCDR, PTUPCDR

Evaluation: leave-last-out on games, full-rank Recall@10.

Dataset

Property	Value (changes from Lesson 3)
Users	14,328 (−28% from L3: 19,880)
Overlap	14,328 (100%)
User Movie minimum	**≥ 10 Movies ratings (was ≥ 5)** ≥ 1 games ratings (same)
Movie · game interactions	388,919 · 58,782 (games −33%)
Movie · game items	39,534 · 9,147
Split	Leave-last-out on games

Results

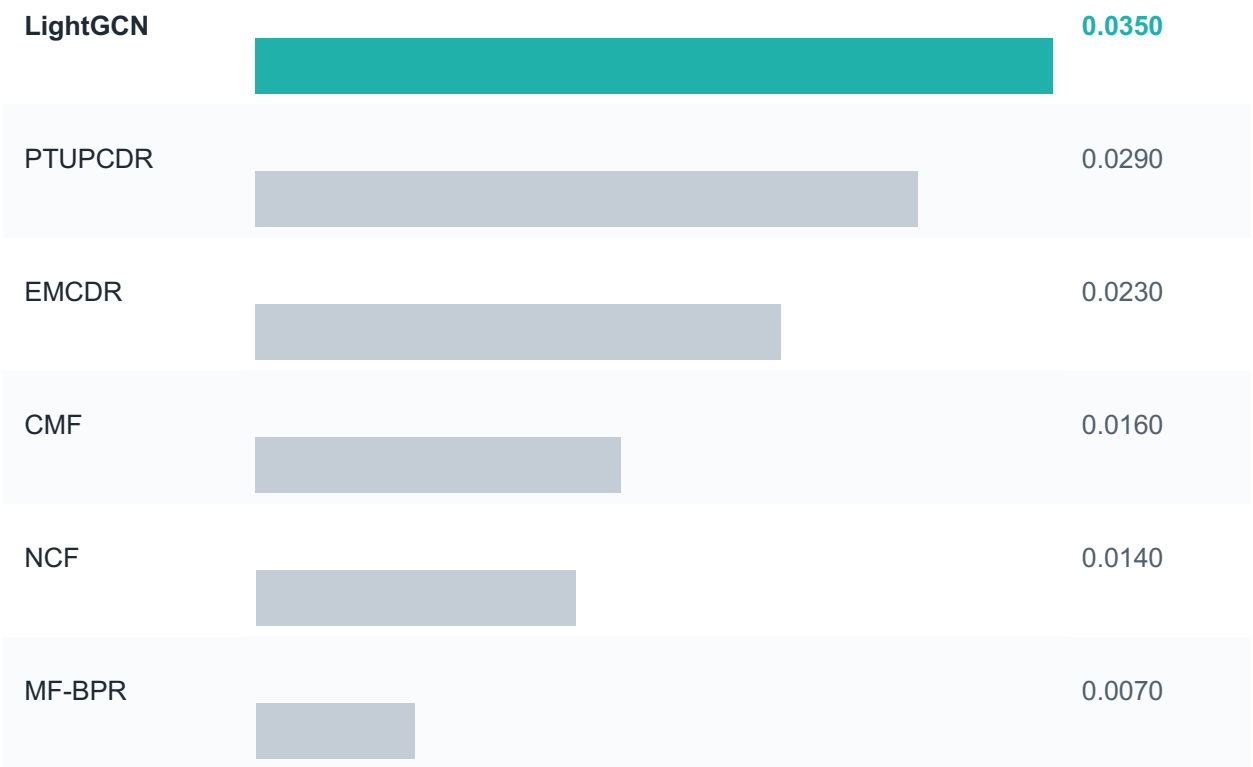
Unfortunately, LightGCN still wins.

But the trend is clearer: **PTUPCDR closes to within 17% of LightGCN** (was 42% in L3)**; EMCDR narrows to −34%; CMF −54%**. LightGCN dropped −37% because its game-heavy users were removed.

CDR is more robust to the filter change — richer movie history compensates for the smaller game side.

Model	Family	Recall@10	NDCG@10	% gap
LightGCN	Graph (single-domain)	**0.0350**	**0.0180**	—
PTUPCDR	Personalized mapping (CDR)	0.0290	0.0154	−17.1%
EMCDR	Mapping (CDR)	0.0230	0.0116	−34.3%
CMF	Joint MF (CDR)	0.0160	0.0078	−54.3%
NCF	Neural (single-domain)	0.0140	0.0073	−60.0%
MF-BPR	MF (single-domain)	0.0070	0.0029	−80.0%

Recall@10 **CDR narrows the gap to LightGCN**



> **KEY FINDING**
>
> **Confirmed —** Richer source embeddings help CDR. PTUPCDR closes the gap from −42% (L3) to −17% (L4). Single-domain models lose ground because the tighter movie filter removes their game-heavy users.

Summary

Lesson 4 raises the movie-history minimum from ≥ 5 to ≥ 10 to test whether source-richness helps CDR close the remaining gap to LightGCN.

The takeaway: CDR's metrics improve as source-to-target signal ratio increases.

Key findings

Finding	Evidence
PTUPCDR nearly matches LightGCN	Gap narrows to **−17%** (was −42% in L3); EMCDR −34% (was −56%)
LightGCN drops most	−37% from L3. A tighter movie filter removes its game-heavy users.
CDR is robust to the filter	PTUPCDR only drops 9% vs L3; EMCDR 6%. Richer movie history compensates for fewer games.
Trend holds across L2–L4	CDR gap to LightGCN: −71% (L2) → −42% (L3) → −17% (L4). The user-filtering lever is paying off.

Reflection

L4's result is a small but surprising one: the population most hurt by a tighter filter is not the model you expect. We indirectly removed game-heavy users and LightGCN dropped more (−37%) than PTUPCDR (−9%). That further reinforces a principle we also observed in past lessons: a model's performance is the reflection of the data characteristic, not an intrinsic property of the model. Change who is in the user group and the metrics changes.

> **NEXT UP**
>
> CDRs performance improved, but **LightGCN still wins**, proving it is a very strong SDR model. We wonder in which scenarios would make CDRs completely outshine LightGCN? Next lesson is the most important one in the project.

5.5 Lesson 5: Cold-Start (Zero Game History)

So as you know so far, **LightGCN still wins**. In the Amazon game domain, it has strong collaborative graph connections among users and items to work on, but what if we give it a dataset where it has **NONE** graph connection to work on?

Every previous lesson used leave-last-out — every test user had SOME games in training history. In this lesson, we change the train/test split: a subset of users have **ALL** their game interactions **HIDDEN** from training.

This is the hardest test for any recommender: predict games for users with zero game history. Single-domain models have nothing to learn from; but will it be the case for CDR? Let's find out.

> **HYPOTHESIS**
>
> *When users have zero game history, do CDR mapping models (EMCDR, PTUPCDR) decisively beat single-domain models?*

Setup

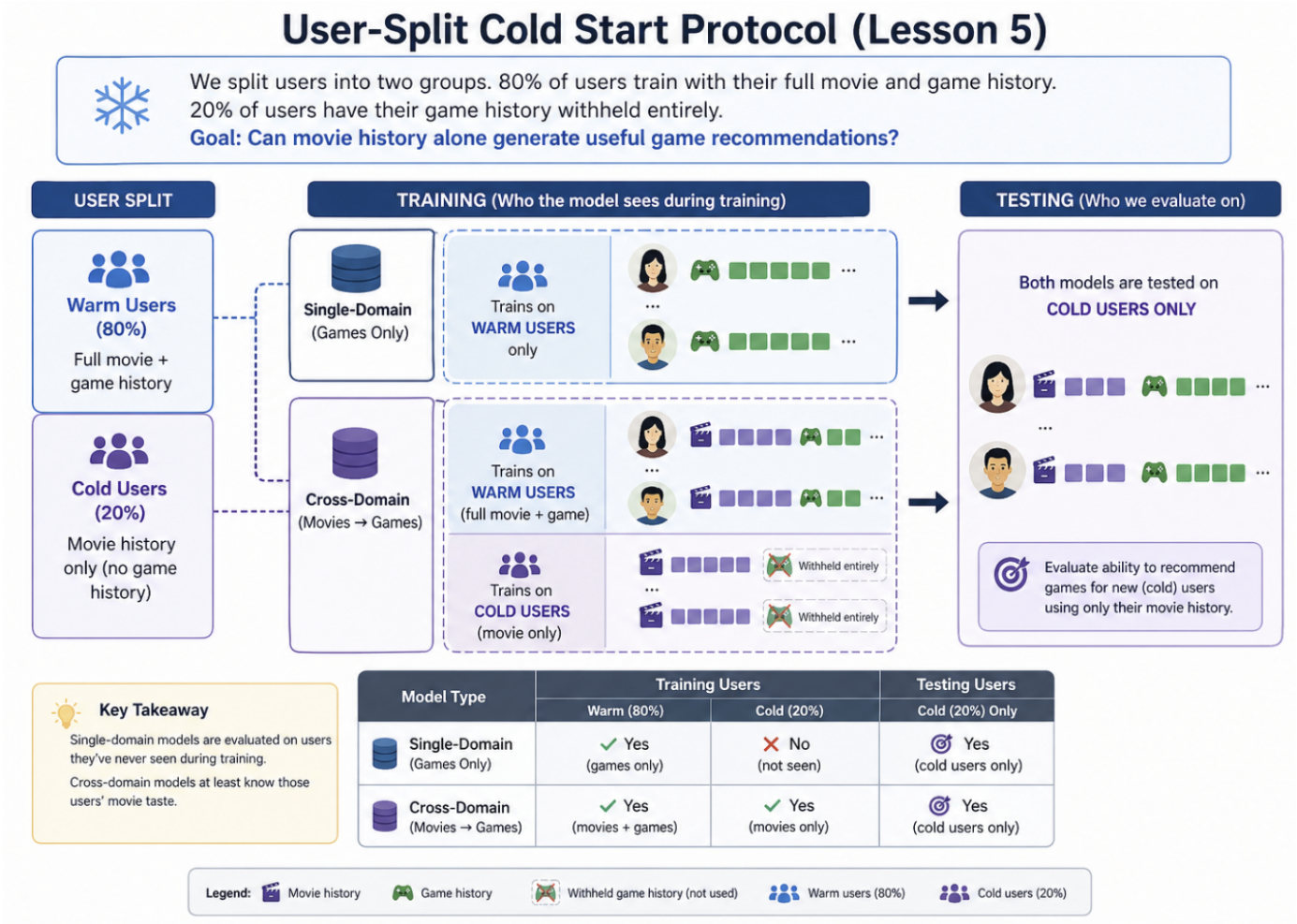
Users are split into two groups:

- **Warm users (80%)**: have both movie and game history.
- **Cold users (20%)**: have movie history only; game history is withheld.

Model Family	Training Protocol	Testing Protocol
Single-Domain (SDR)	• Train only on **warm users' game history**. • Do **not see cold users** during training.	Test on cold users' withheld game history ⇒ Performance should be similar to **blind** guesses.

Cross-Domain (CDR)	• Train on **warm users' movie + game history**. • Also see **cold users' movie history** during training.	Test on cold users' withheld game history ⇒ Model learns **signals from movie domain**.

Check the illustration below to understand better:



Key idea:

- Both models are tested on cold users, but only cross-domain models know their movie preferences. This measures whether movie signals can help recommend games.

Evaluation: user-split cold-start, Recall@10 on cold users only.

Dataset

Property	Value
Source dataset	processed_overlap (L3 subgroup, movies ≥ 5)
Eligible users (games ≥ 2, movies ≥ 5)	12,654
Warm users (train)	10,124 (80%)
Cold users (zero games in train)	2,530 (20%)
Evaluation users (capped)	2,530
Movie interactions (all users)	430,736
Game interactions (warm only)	64,057
Split	User-split cold-start 80/20

Results

The ranking flips completely. **All three CDR models beat every single-domain collaborative model:**. Popularity is the strongest single ranker (0.0381) — cold users often pick well-known first games. CDR mapping approaches Popularity, which is the correct regime for cold-start.

Model	Family	Recall@10	NDCG@10	× LightGCN
EMCDR	Mapping (CDR)	**0.0334**	0.0152	**3.3×**
PTUPCDR	Personalized mapping (CDR)	**0.0301**	0.0124	**3.0×**
CMF	Joint MF (CDR)	**0.0210**	0.0112	**2.1×**
LightGCN	Graph (single-domain)	0.0100	0.0062	1.0×

NCF	Neural (single-domain)	0.0020	0.0008	0.20×
MF-BPR	MF (single-domain)	0.0007	0.0002	0.07×

Recall@10 **CDR dominates cold users · Popularity leads**

KEY FINDING

Confirmed — On cold users (zero game history), every CDR model beats LightGCN by 2–3×, and every single-domain MF/neural model collapses to near-zero. This directly justifies the routing rule: route zero-game users to CDR — not to any model that relies on target-domain interactions.

Summary

Lesson 5 withholds all game interactions for cold users and asks every model to rank the complete cold start users on games.

Every CDR model beats every single-domain collaborative model. Single-domain MF and neural models (MF-BPR, NCF) collapse to near-zero because they have no cold-user game signal to learn from.

The takeaway: CDR is not just helpful for cold users, it is **essential**. Route zero-game users to EMCDR / PTUPCDR / CMF / Popularity — not LightGCN, NCF, or MF-BPR. This is the CDR-essential half of the routing rule we finalise in.

Key findings

Finding	Evidence
CDR is essential for cold users	Every CDR model beats LightGCN: **EMCDR 3.3×, PTUPCDR 3.0×, CMF 2.1×**. MF-BPR and NCF collapse.
Single-domain CF collapses	MF-BPR = 0.0007, NCF = 0.0020. No game signal → essentially random guesses.

Reflection

Lesson 5 is the moment we prove the CDR's reason to exist. It shows that in the toughest cold-start situation—when users have absolutely zero game history—our powerful single-domain LightGCN collapses. Only the Cross-Domain Recommenders (CDR) can translate movie signals into a meaningful game recommendation, which is exactly the job they were designed for. But we must remember LightGCN still dominates when a user is warmed up.

So, the real takeaway is not that one model is *better* than the other. Instead, Lesson 5 gives us the critical insight for designing an intelligent system: **it is all about choosing the right model for the right user segment**.

> **NEXT UP**
>
> L5 showed **CDR wins at zero-game cold-start,** but their inner working mechanisms are rather complicated and require extensive training.
>
> I still remember in the Day 4-5 of IRS module lectures, I learned about 3 main categories of recommendations: Collaborative Filtering group, Content-aware group,and Market basket analysis group. In our current models, they are in Collaborative Filtering group, let’s explore Content-aware group in the next Lesson.

5.6 Lesson 6: Content-Aware CDR (SBERT)

Every model so far is collaborative — it learns from interactions. Collaborative filtering is systematically blind to long-tail items that few users have touched.

Lesson 6 adds two content-based models that read the text of each item (title + description) through a pretrained sentence-BERT encoder. No training on interactions — pure semantic similarity.

> **HYPOTHESIS**
>
> *Can content-based content models match the best collaborative models overall?*

Setup

We use the same 14,328-user source-rich user group from L4, add two new content models, and compare standard leave-last-out Recall@10 overall and broken down by **user-item subgroup** (warm vs cold).

- **Content (training-free)** — SBERT (single-domain, game-side), SBERT-CDR (uses movie titles too)
- **Collaborative benchmarks** — LightGCN (graph), PTUPCDR (CDR mapping)

Evaluation: leave-last-out on games; SBERT encoder is all-MiniLM-L6-v2 (384-dim).

Dataset

Property	Value (same as Lesson 4)
Users (100% overlap)	14,328
Movie · game items	39,534 · 9,147
Movie · game interactions	388,919 · 58,782
SBERT encoder	all-MiniLM-L6-v2 (384-dim)
Split	Leave-last-out on games

Results

We did not have high hopes for this, because the content-aware models are just simple word to vector conversion and they do not have any training at all. How could they perform better than the highly researched SDR and CDR?

But we were wrong! SBERT-based models match the best collaborative model overall. **SBERT-CDR reaches 0.0330 — within 1.5% of LightGCN; plain SBERT is 0.0310 with zero training**. Both beat PTUPCDR (0.0220). Content models are now a first-class option.

Model	Family	Recall@10	% of LightGCN
LightGCN	Graph (single-domain)	**0.0335**	100%
SBERT-CDR	Content (CDR, no training)	**0.0330**	**98.5%**
SBERT	Content (single-domain)	0.0310	92.5%
PTUPCDR	Personalized mapping (CDR)	0.0220	65.7%

> **KEY FINDING**
>
> **Confirmed —** SBERT-CDR matches LightGCN overall (98.5%) while **SBERT** has lightly poorer**,** but still **good** performance.

Summary

Lesson 6 introduces two training-free content models: SBERT (single-domain, game titles only) and SBERT-CDR (blends game-side SBERT with movie-side SBERT). Both use a pretrained sentence encoder (all-MiniLM-L6-v2, 384-dim).

The takeaway: add content signals to the list of algorithms to use. SBERT / SBERT-CDR fills that gap and makes the system robust across the full popularity spectrum.

Key findings

Finding	Evidence
SBERT alone can work well, but **SBERT-CDR** performs better for movie rich-game poor user group	**SBERT-CDR** (0.0330) **>** **SBERT**(0.0310)

SBERT-CDR ≈ LightGCN overall	0.0330 vs 0.0335 (−1.5%). Zero training; pure semantic similarity.

Reflection

This lesson was a big surprise for us because it challenged the idea that we always assume that more complex models are always better. We had spent so many hours training and tuning those collaborative models like LightGCN and PTUPCDR, only to see a completely training-free approach—just reading the item descriptions with SBERT—come out and almost match LightGCN overall.

This proves an important point for anyone designing a system: we don't always need the most complicated, latest model architecture. Sometimes the simplest tool is the most efficient.

> **NEXT UP**
>
> So far we have explored content-aware, collaborative filtering approaches, how about Market Basket Analysis) recommendation approach?

5.7 Lesson 7: Co-occurrence Reranking

We asked ChatGPT on how we can bring in the idea of Market Basket Analysis into our cross-domain algorithm, and it suggested to us an interesting co-occurrence counting method! Furthermore, we can use this as another re-ranking in the final scoring layer, after the main model's recommends us a candidate list

The construction is straightforward: for every (movie, game) pair, we calculate the number of co-likers, normalize this by the mean of their respective audience sizes, and apply a weighted bonus to the base model's score. This approach mirrors the classic Amazon's "**customers who bought this also bought...**.

> **HYPOTHESIS**
>
> *Does a simple training-free co-occurrence rerank applied on top of every base model produce consistent lift across LLO, cold-start, and every model family?*

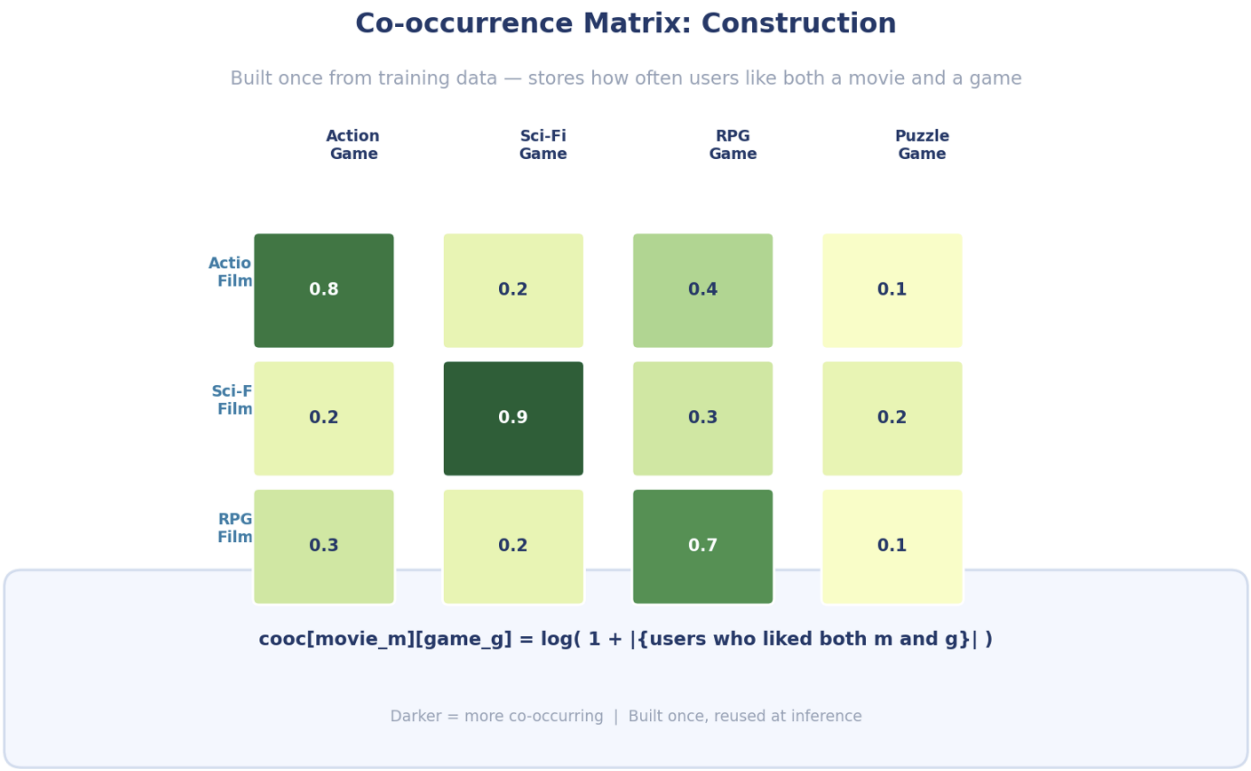
Co-occurrence Matrix

For every (movie, game) pair, we count how many users rated both. This count is normalized by the mean of each item's total user base, creating a co-preference score.

$$c_{m,g} = \frac{|\mathcal{U}_m \cap \mathcal{U}_g|}{\sqrt{|\mathcal{U}_m|\,|\mathcal{U}_g|}}$$

Equation 8: Co-occurrence matrix and reranking score.

The co-occurrence matrix C_{m, g} is the normalized overlap of users who liked movie m ($\mathcal{U}$_m) and users who liked game g ($\mathcal{U}$_g):



Co-occurrence Reranking

Co-occurrence reranking is applied as a post-processing layer on top of any base model's scores using the following formula, where λ is the re-rank weights:

$$s_{u,g}^{\text{rerank}} = s_{u,g}^{\text{base}} + \lambda \sum_{m \in \mathcal{I}_u^{\text{movie}}} c_{m,g}$$

For a given user, we first obtain the base model's score for each candidate game. Then, for each game, we sum the co-occurrence values over all movies the user has watched, producing a co-occurrence

bonus. This bonus is scaled by a tunable strength parameter λ and added to the base score. The final ranking is by combined score.

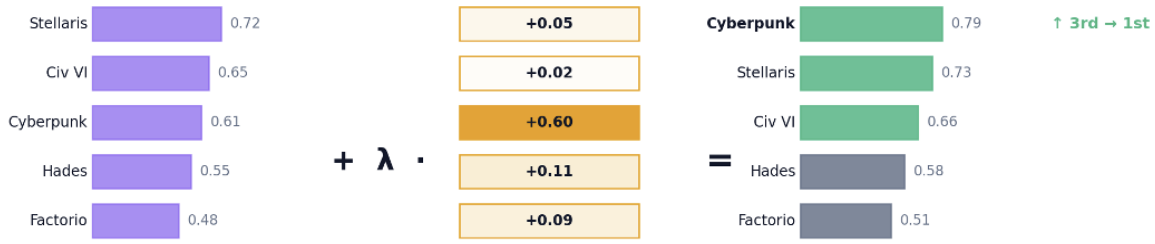


Figure 19: Co-occurrence reranking at inference — base model scores are boosted by cross-domain signals

Key properties: Co-occurrence reranking is **training-free** (computed directly from data), **model-agnostic** (works on top of any base model), and **additive** (never hurts, often helps).

Setup

We compute one co-occurrence matrix c[m,g] once from the training interactions, then for every model we rerank its scores as **s_rerank(u,g) = s_base(u,g) + λ · Σ_{m ∈ user-movies} c[m, g]**. No model is retrained. λ is tuned once (λ = 0.05) and fixed across all experiments.

- **Base models** — MF-BPR, NCF, LightGCN, CMF, EMCDR, PTUPCDR, BiTGCF — every model from L1–L7
- **Deployments tested** — L3 LLO (100% overlap), L5 cold-start, plus two extra regimes

Evaluation: leave-last-out on games for LLO; user-split for cold-start; Recall@10.

Dataset

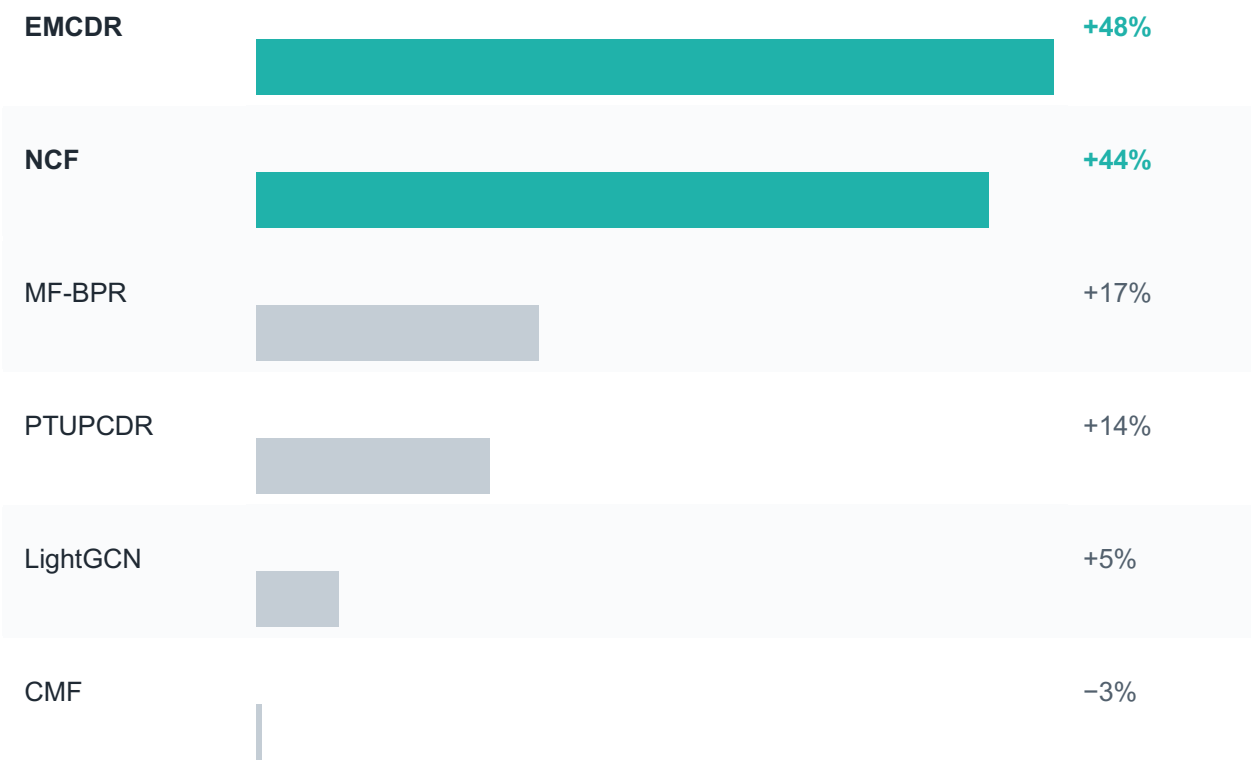
Property	Value
New models trained	None — post-processing layer only
Matrix formula	$c_{m,g} = \frac{\lvert\mathcal{U}_m \cap \mathcal{U}_g\rvert}{\sqrt{\lvert\mathcal{U}_m\rvert\,\lvert\mathcal{U}_g\rvert}}$
Blend weight λ	0.05 (tried 0.01, 0.05, 0.1, 0.2)
Evaluation group	L3 LLO + L5 cold-start
Training cost	Zero — matrix computed once from training data

Results (LLO, Lesson 3 user group)

Co-occurrence reranking improves every model except CMF. **Biggest wins: EMCDR +48%, NCF +44%, BiTGCF +25%, MF-BPR +17%**. The effect size is inversely proportional to how well the base model already used movie signal — weak CDR models gain the most.

Model	Base Recall@10	+Cooc Recall@10	Δ
LightGCN	0.0595	**0.0625**	+5%
EMCDR	0.0230	**0.0340**	**+48%**
NCF	0.0260	**0.0370**	**+44%**
MF-BPR	0.0450	**0.0520**	+17%
PTUPCDR	0.0320	**0.0360**	+14%
CMF	0.0400	0.0380	−3%

Cooc Δ · LLO Recall@10 **Weakest CDR gains the most**



Results (cold-start, Lesson 5 user group)

The rescue effect is even larger at cold-start. **LightGCN jumps from 0.0080 → 0.0310 (+289%); EMCDR +31%. CDR mapping models (which already use movie signal) change little — cooc is additive where interactions were missing**.

Model	Cold Recall@10	+Cooc Recall@10	Δ
LightGCN	0.0080	**0.0310**	**+289%**
EMCDR	0.0300	**0.0395**	**+31%**
PTUPCDR	0.0305	0.0330	+8%

> **KEY FINDING**
>
> **Confirmed —** Co-occurrence reranking is a universal free lift. It improves every model family in every regime tested, with the largest gains on the weakest base models (EMCDR +48% LLO, LightGCN +289% cold-start). Cost: zero training. It becomes the default post-reranker for every row our Recommender UI Portal

Summary

Lesson 7 adds a single layer: precompute one (movie × game) co-occurrence matrix, then rerank every base model's output by s_rerank = s_base + λ · Σ cooc[m, g] over the user's movies.

The takeaway: co-occurrence rerank is the cheapest tool in the toolbox. It should be applied by default on every base model output because it either helps materially or leaves performance essentially unchanged.

Key findings

Finding	Evidence
Universal lift	Every model except CMF gains **+5% to +48% on LLO** — weaker models gain more
Cold-start rescue	LightGCN **0.0080 → 0.0310 (+289%)** — item-item signal fills in when user-item signal is missing

Zero cost	Matrix computed once; λ = 0.05 fixed; adds microseconds per request.

Reflection

This whole co-occurrence reranking layer was honestly one of the coolest things we figured out(but the idea was suggested by ChatGPT, we need to give it credit!). We were spending so much time struggling with the really complicated models (the collaborative and mapping ones), but then this super easy counting method—just looking at what movies and games people liked together—gave almost every model a huge double-digit boost for practically no effort!

It taught us a big lesson: **you don't always need the fanciest new algorithm**. Sometimes, just looking at the simple stuff in the data, the basic patterns of what people do, or **a basic concept that you learned in class** (Market Basket Analysis), can give you the best ideas that even the smart machine learning models miss. Now, we use this quick, effective layer first because it's a guaranteed way to improve things, and it even helps us figure out if our main models aren't trained properly.

5.8 Lessons Recap

Taken together, the eight lessons form a deliberate staircase: each step removes one variable the previous step could not explain. The result is a much clearer picture of when cross-domain recommendation helps.

The Learning Journey Reflections

Initially prior to all the benchmarking, when we were reading about the CDR inner working mechanism, we were very fascinated, like a child just discovering new cool stuffs. Our initial excitement over complex Cross-Domain Recommendation (CDR) algorithms quickly evaporated, as I learned they fail horribly when there is a lack of bridge users.

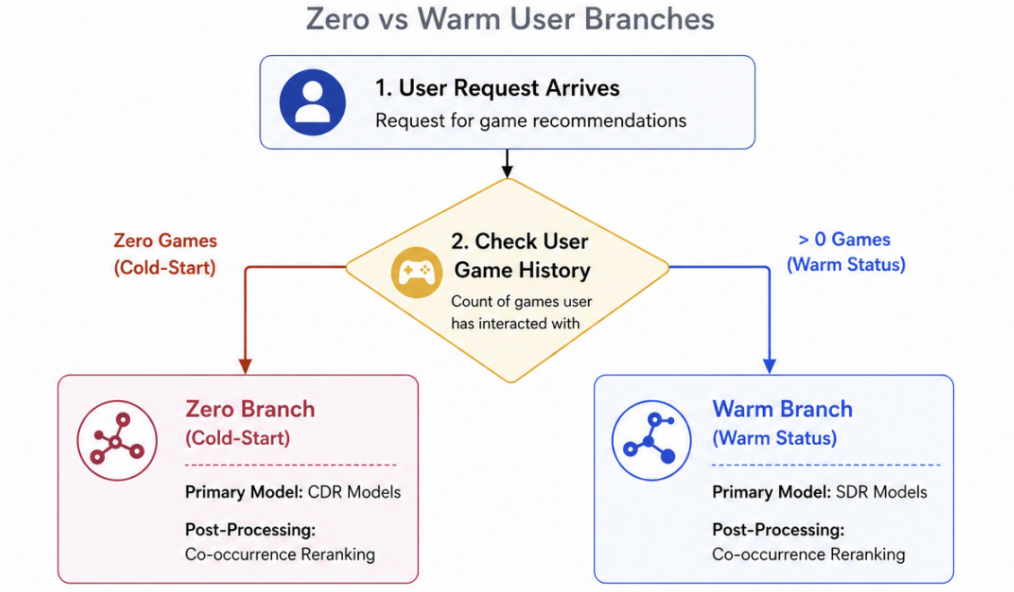
Beyond the algorithm internals, an equally important lesson was that a handful of easy-to-overlook surrounding details — the choice of loss function, increase the bridge users, finding suitable user subgroups, the train/test split... — often decide whether CDR shines or stalls. Across the lesson series we systematically isolated each of these conditions and verified its impact one experiment at a time, validating that the small, "boring" decisions were doing more for CDR than the architecture choice itself.

Most importantly, the whole project taught me the immense power of **simplicity**. After spending weeks tuning complex deep learning architectures, the ultimate performance booster was the simple, training-free **SBERT content-aware (L6)** and **Co-occurrence Reranking layer (L7)**. This single post-processing step provided a universal lift for almost every model (up to +48% LLO) and rescued LightGCN from cold-start failure (+289%).

Another breakthrough was realizing that the optimal model is entirely **context dependent**. CDR is **essential** for cold-start users with zero game history (beating LightGCN by 2–3×), whereas Single-Domain Recommenders (SDR) like LightGCN win decisively for "warm" users with a history of a few

games. This clear performance split justifies the need for a **Routing Decision Rule** to match each user to the most effective algorithm.

The Routing Decision Rule



8. System Design

Beyond the experimental benchmarks, we built a full-stack demonstration system that brings the research findings to life as an interactive web application. The demo exposes the hybrid recommender built from the lessons — segment-routed models, universal co-occurrence reranking, SBERT-CDR path — so collaborative, cross-domain and content-based approaches can be compared side by side on real users.

8.1 High-Level Architecture

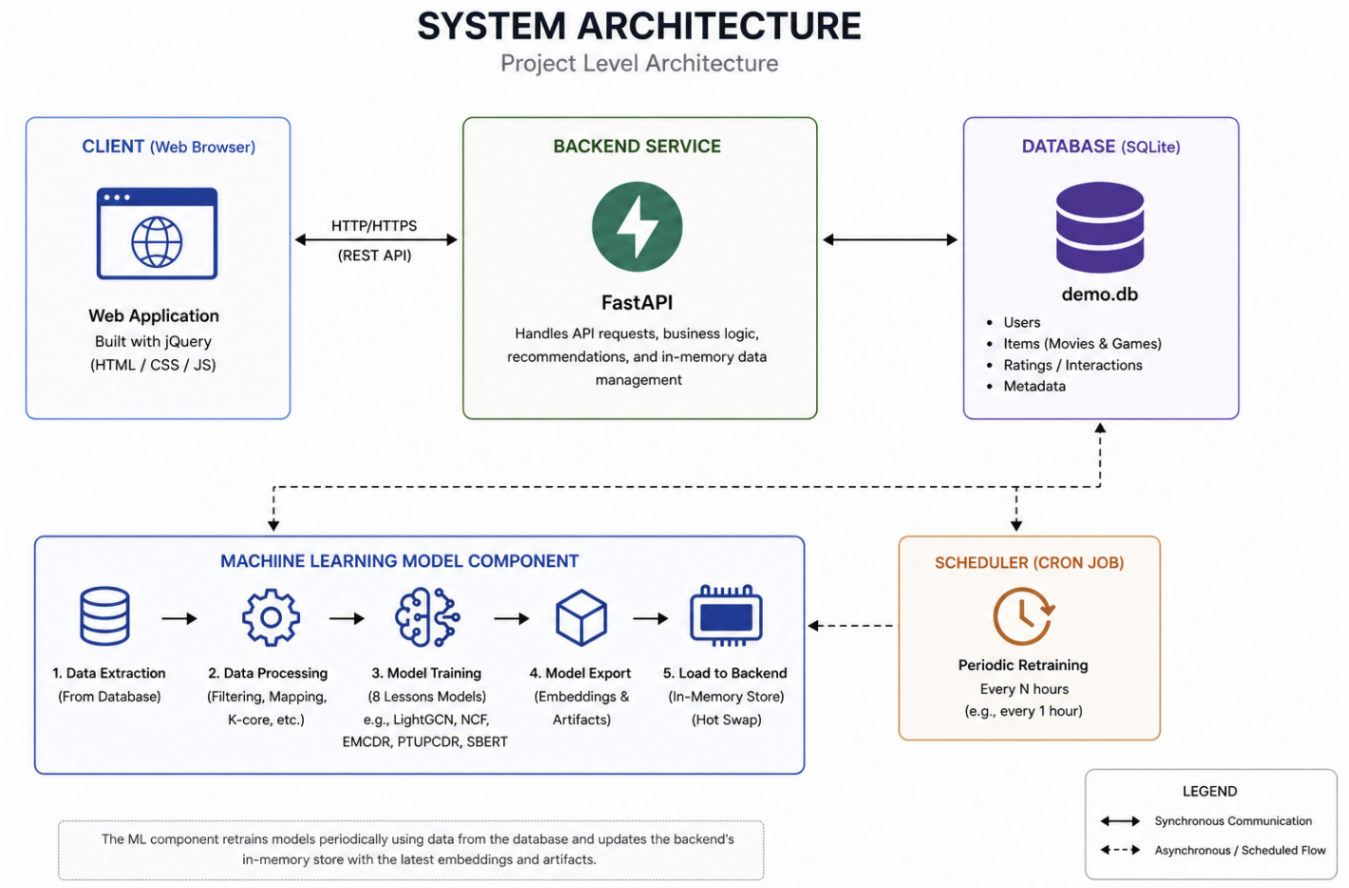


Figure 32: System architecture

Component	Role	Implementation notes
Client	Renders UI, rating widgets, multi-row recommendation feed; no scoring logic.	Static index.html; jQuery SPA; calls REST endpoints (/api/recommendations, /api/ratings, /api/users) over HTTP/HTTPS.
Backend	Handles every API request and the entire business logic: segment classification, row routing, embedding lookup, scoring, co-occurrence rerank, deduplication, JSON serialisation.	FastAPI process; loads model artifacts into in-memory NumPy store at startup; sub-millisecond scoring.

Database	Persistent state — user accounts, items (movies and games metadata), ratings/interactions, auxiliary tables.	Single WAL-mode SQLite file (demo.db); no Postgres, no ORM migrations.
ML component	Offline pipeline producing the embedding artifacts the backend serves from. Runs out-of-process and decoupled from request handling.	Five stages: data extraction → processing → model training → export → in-memory boost. Outputs NumPy .npy files.
Scheduler	Periodically triggers the ML component so the exported artifacts catch up with newly written ratings.	Cron job; default cadence every 1 hour; backend reloads new artifacts at the next request.

8.3 User Journey & Recommender Routing

How the demo backend chooses between Single-Domain (SDR) and Cross-Domain (CDR) recommenders, and how the experience evolves as a new user rates items.

8.3.1 TL;DR

Game ratings	Segment	Routed to	Why
0	cold_start	**CDR rows** (CMF, EMCDR, PTUPCDR)	No collaborative game signal — transfer movie taste into game space.
≥ 1	warm	**SDR rows** (LightGCN, MF-BPR, NeuMF)	Game collaborative signal exists — use direct game-space models.

SBERT-CDR rows are shown in both segments — they are always sourced from movie ratings to recommend semantically similar games.

8.3.2 Routing in detail

Cold-start path (0 game ratings)

Rendered rows, in order:

1. **CMF + Co-occurrence** — "Where Your Movies Meet Games"
2. **EMCDR + Co-occurrence** — "Translated From Your Movie Taste"
3. **PTUPCDR + Co-occurrence** — "Personalized From Your Movie Profile"
4. **SBERT-CDR Hidden Gems** — "Hidden Gems Matching Your Movies"

A cold-start user with **zero ratings** sees no rows. Once they rate one or more movies, CDR rows appear.

Warm path (≥ 1 game rating)Warm path (≥ 1 game rating)

Rendered rows, in order:

1. **SBERT** — "Based on your 5 most recent games" — pure SBERT single-domain, ranks games by cosine similarity to the user's last 5 liked games
2. **LightGCN + Co-occurrence** — "Players Like You Also Loved"
3. **MF-BPR + Co-occurrence** — "Top Ranked by Similar Players"
4. **NeuMF + Co-occurrence** — "Neural Network Picks"

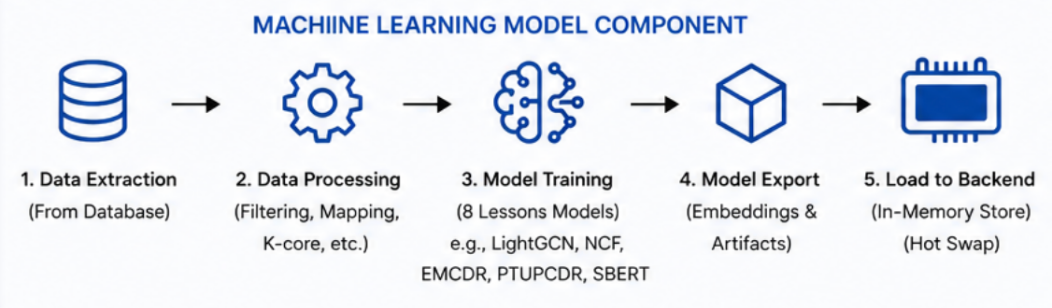
The New user's first game rating **flips** the segment from cold_start to `warm` and the row stack changes accordingly.

What a new user sees, step by step

Action	Segment	Rows that appear
Just registered, no ratings	cold_start	None (no signal to transfer)
Rates 1+ **movies**	cold_start	CMF (Where Your Movies Meet Games) EMCDR (Translated From Your Movie Taste) PTUPCDR (Personalized From Your Movie Profile) SBERT-CDR Hidden Gems Matching Your Movies Co-occurrence rerank applied on every learned-model row.
Rates 1+ **games** (with or without prior movies)	warm	SBERT (Based on your 5 most recent games) LightGCN (Players Like You Also Loved) MF-BPR (Top Ranked by Similar Players) NeuMF (Neural Network Picks) Co-occurrence rerank applied on every learned-model row.
Returns later (after server restart)	unchanged	Same — ratings persist in SQLite and are restored to memory at startup with their domain re-attached from the item catalog.

8.3 ML Pipeline

The bottom band of *Figure 32 System architecture* is the offline ML component. It runs out-of-process and produces the embedding artifacts that the backend serves from. The pipeline has five deterministic stages, run in order:



1. Data extraction — pull the latest users, items, and ratings tables from demo.db. The extraction step talks to the same SQLite file the backend writes to, so newly added ratings show up on the next pipeline run.

2. Data processing — apply the subgroup filters with implicit conversion at rating ≥ 4, user k-core ≥ 10, overlap filter (movies ≥ 5 / games ≥ 1), and the post-overlap movie popularity filter (≥ 10 ratings within the overlap subset). The cleaned interactions are exported to Parquet.

3. Model training — train each model in the portfolio in parallel: MF-BPR, NCF, LightGCN, CMF, EMCDR, PTUPCDR, and SBERT/SBERT-CDR. Hyperparameters come from Table 8 (Chapter 6). Each training run writes its tensorboard log so a regression in any one model is visible without having to inspect the embedding output.

4. Model export — save the artifacts every model needs to score at inference: U and V embedding matrices for the collaborative models, the per-expert weights for PTUPCDR plus the gate, the SBERT item-embedding matrices, and the (movie × game) and (game × movie) co-occurrence matrices. All artifacts are written as NumPy .npy files into a single artifacts/demo directory.

5. In-memory load — at server startup the backend memory-maps every .npy file from artifacts/demo into the in-memory embedding store. From that point on every recommendation request only does dot products against in-memory tensors, so a request never touches the disk.

8.4 Refresh Architecture

A scheduler (cron job, top-right of Figure 32) triggers the ML training pipeline on a fixed interval (default: every hour). Between scheduled runs the backend keeps serving from the most recent in-memory snapshot; when a new pipeline run completes, the backend reloads the new artifacts at the start of the next request. This keeps the request path completely decoupled from training — a long retraining run never blocks recommendations.

Within a single in-memory snapshot, an end-user-added rating still propagates to recommendations almost immediately, via two refresh tiers:

Tier 1 (instant, few ms) – refreshed after a user rate new item:

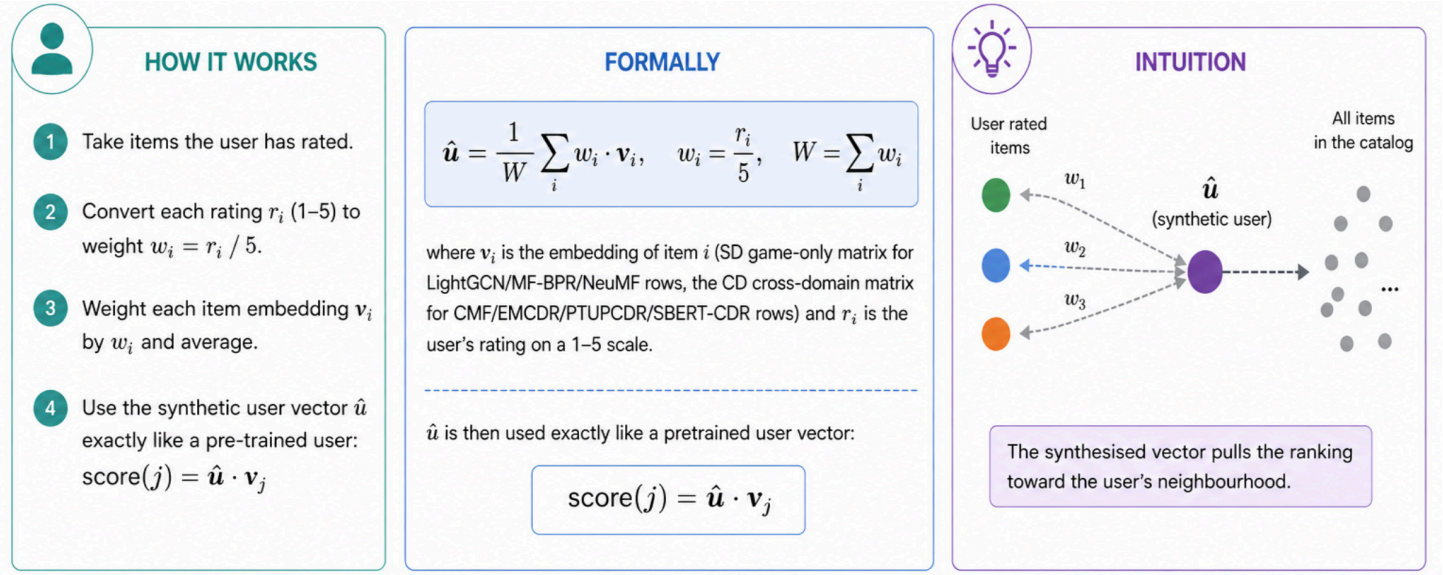
- Training-free row —SBERT(+co-occurrence rerank) — read the latest rating directly from SQLite and recompute their score on each request. No learned weights are involved, so a new rating appears in those rows on the next click.
- For new users that have not been put in the training before(thus no user embeddings yet): Learned-model rows — MF, NeuMF , LightGCN, CMF, EMCDR, PTUPCDR,— read pre-computed item embeddings from the in-memory store and temporarily synthesize user's vector by averaging out the latest item's embeddings that user has ranked by rating weight(see below for mechanism). Then for each recommendation row, we use dot product of the synthesized user's embedding to item's embedding to recommend items.

Tier 2 (slow, runs every hour): full retraining of the six collaborative models — LightGCN, MF-BPR, NeuMF (single-domain) and CMF, EMCDR, PTUPCDR (cross-domain). A background scheduler (retrain.py, in-process daemon) runs hourly: it reads the latest user ratings, re-fits all six models, and hotswaps the new embeddings into the live store — no disk writes, no request blocking. SBERT and the co-occurrence matrix sit outside this tier because they don't depend on ratings.

The combination is intentional: **instant freshness** on the cheap rows the user notices first, and **periodically hourly refresh** of the expensive learned models. If a deployment needs faster model freshness the scheduler interval can be shorten without affecting the running system— nothing in the request path or the storage layer changes.

8.4.1 Fold-in: synthesising a user vector from rated items

What it does. Fold-in builds a synthetic user embedding on the fly from the user's rated items, instead of looking it up in a pre-trained user matrix. Note: this logic is only for cold start users whose user embeddings are not found (have not been trained by the scheduled cron-job)



Why we do it.

• **Cold-start coverage.** Brand-new users have no row in the pretrained user matrix; the usual u[idx] lookup returns nothing. Fold-in produces a usable vector in microseconds — no training, no waiting for the next scheduler tick.

• **Within-snapshot freshness.** Between hourly scheduler ticks the trained weights are frozen, but the rated set is read live from SQLite at every request. Fold-in is what makes "rate one game → see different recs on the next click" feel real for users without a stale pre-trained slot.

When it's most useful:

• Just-registered accounts whose first session has no pretrained vector to fall back on.

• Cold-start CDR scenarios — users with only movie ratings, where fold-in over CD movie embeddings is the only path into game space (CMF/EMCDR/PTUPCDR + SBERT-CDR Hidden Gems all depend on it).

• Returning warm users created mid-cycle: they exist in user ratings but not yet in the latest pretrained user matrix; fold-in keeps their recommendations sensible until the next retrain.

9. Conclusion and Future Work

Conclusion

Our experiments show that cross-domain recommendation from movies to games works under specific conditions: high user overlap (≥80%), sufficient source domain history (≥10 ratings), and particularly at cold-start where users have no target-domain interactions. The optimal strategy is not a single model but a routing rule that matches the model to the user's context.

The co-occurrence reranking layer is the most practically impactful finding — it requires no training, operates at test time, and consistently improves every model. It also serves as a diagnostic tool for detecting CDR model misconfiguration.

Future Work

- Additional domain pairs: Test the routing rule on other Amazon category pairs (e.g., books→movies, electronics→games)
- Online evaluation: Deploy the routing rule in an A/B test to measure real-world click-through improvements
- Hybrid model: Train a single model that combines LightGCN graph signal with SBERT content and cooc behavioral transfer
- Temporal dynamics: Investigate whether user preferences transfer differently over time (e.g., movie taste from 2020 may not predict 2024 game preferences)

10. References

https://medium.com/data-science-in-your-pocket/recommendation-systems-using-neural-collaborative-filtering-ncf-explained-with-codes-21a97e48a2f7

He, X., Deng, K., Wang, X., Li, Y., Zhang, Y., & Wang, M. (2020). LightGCN: Simplifying and powering graph convolution network for recommendation. SIGIR 2020.

https://medium.com/@jn2279/better-recommender-systems-with-lightgcn-a0e764af14f9

https://github.com/RUCAIBox/RecBole-CDR

[3] Singh, A. P., & Gordon, G. J. (2008). Relational learning via collective matrix factorization. KDD 2008.

https://github.com/USTCAGI/Awesome-Cross-Domain-Recommendation-Papers-and-Resources

https://medium.com/@bricesh/nice-classification-recommendation-using-sentence-bert-b1af32d0131e

https://github.com/easezyc/WSDM2022-PTUPCDR

https://www.evidentlyai.com/ranking-metrics/precision-recall-at-k

AI chatbots: ChatGPT, Claude, Google Gemini

Appendix A: Project Proposal

Date of proposal: 21st Mar 2026

Project Title: Single-Domain and Cross-Domain Recommendation System for Movies and Video Games

Group Name: **21**

Member List:

Vo Minh Khoi A0339229W

Sponsor/Client:
Institute of Systems Science (ISS)
National University of Singapore (NUS)
25 Heng Mui Keng Terrace, Singapore

Background:

Recommendation systems play a key role in many digital platforms such as Netflix, Amazon, and Spotify. These systems analyze user behavior both explicitly and implicitly to suggest relevant content, helping users discover new items more easily and keeping them engaged more with the platform.

Most recommendation systems operate within a **single domain**. For example, a movie platform recommends movies based only on movie watching/rating history, and similar for game platforms. However, many users usually enjoy **different types of entertainment**. A person who enjoys watching movies may also spend time playing games, listening to music, or reading books...

Because of this common behaviour, it is increasingly valuable to explore recommendation systems that can learn user preferences across different domains. This is particularly relevant today as entertainment platforms begin expanding beyond their original content categories. For example, **Netflix has recently started offering games on its platform**, in efforts to encourage users to spend more time within the platform.

This project explores how each **single-domain** and **cross-domain recommendation approach** can improve recommendation quality and help users discover new forms of entertainment.

Aims:

The main aim of this project is to design and build a **recommendation system** on both **single domain** and **cross domain,** capable of recommending both movies/TV content and video games.

The system will implement and compare both **single-domain recommendation algorithms** and **cross-domain recommendation algorithms**. By analyzing user interactions both **within a single domain** and **across two domains**, the project will investigate whether knowledge learned from one domain can improve recommendation quality in another and whether single domain approaches might simply perform better than cross domain counterparts.

Dataset:

This project will use publicly available datasets from the Amazon Product Review Dataset(2023).

Two domains will be selected:

- **Movies and TV**
- **Video Games**

System Workflow:

The workflow of the system is as follows:

1. A user selects an account from a list of sample accounts. Users also can register and log into a new account.
2. The user rates movies or games.
3. The rating data is stored in the system database.
4. The recommendation engine processes user preferences.
5. The system generates recommendations using both:
    - Single-domain algorithms
    - Cross-domain algorithms
6. The recommendations are displayed on the frontend.

This architecture enables easy experimentation and comparison between different recommendation strategies.

Appendix B: Techniques and Skills (MR, RS, CGS)

Course Module	Techniques and Skills
Machine Reasoning	Matrix Factorization with BPR (MF-BPR) Neural Collaborative Filtering (NCF) LightGCN (graph convolutional network) Cross-Domain Embedding Mapping (EMCDR — learned MLP bridge) Personalized Transfer with hypernetwork (PTUPCDR) Sentence-BERT embeddings + cosine similarity
Reasoning Systems	Movie → game co-occurrence rerank Two-lane Routing Decision Rule (explicit rule: 0 games → CDR; ≥ 1 game → SDR) Category-path filtering to identify "actual games" vs accessories/consoles (symbolic rule on the Amazon category string) Item deduplication of DVD / Blu-ray / platform variants via metadata rules
Cognitive Systems	User segmentation (cold-start / warm) Recommendation System Frontend + Backend UI Portal

Appendix C: Setup Guide

The repository ships with a Makefile that drives both workflows. Each Make target maps to the equivalent shell command shown alongside, so the steps can also be run by hand.

Prerequisites

Path	Required tools
Local	Python 3.11+, pip, venv, git
Docker	Docker 24+ (Desktop or CLI). No local Python required.

Option 1 — Local (non-Docker)

A three-step bring-up; the all-in-one target chains them in order.

Step	Make target	What it does
1. Install dependencies	make install	Creates .venv/ and installs the full Python stack (ML + demo).
2. Export model artifacts	make export	Materialises NumPy embeddings, the item catalog, and co-occurrence tables under artifacts/demo/.
3. Run the demo	make dev	Starts the FastAPI backend with hot-reload.
All-in-one	make all	~ make install and make dev

After the server boots**, open** http://localhost:8000

Option 2 — Docker

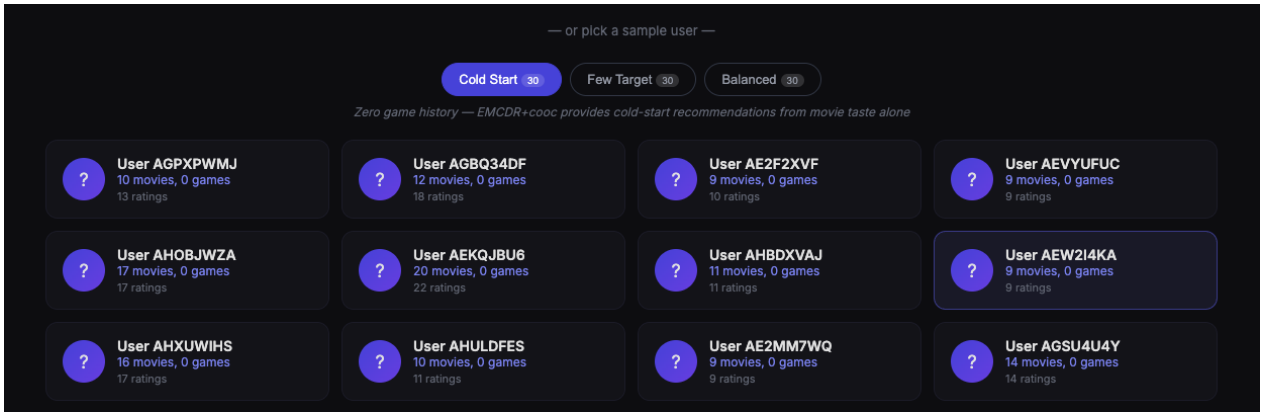
The image is CPU-only (~1 GB), ships pre-built artifacts under artifacts/demo/, and uses requirements-docker.txt — a slimmed-down dependency set that skips ML-only packages such as cornac, matplotlib, seaborn, and pytest.

Step	Make target	Equivalent shell command	What it does
1. Build image	make build	docker build -t crossrec .	Builds the image from the project Dockerfile.
2. Run container	make run	docker run -d --name crossrec-app -p 8000:8000 crossrec	Runs detached and maps host port 8000 → container 8000.

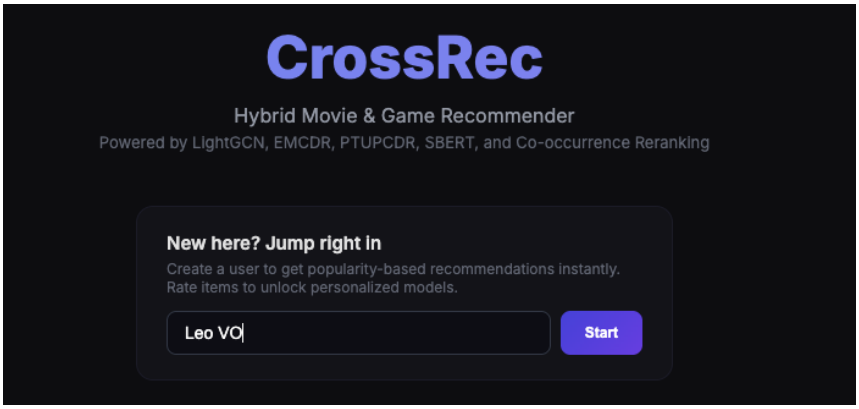
After the server boots**, open http://localhost:8000.**

Verification

After either path, the home page should load with four sample-user tabs (Cold Start, Few Target, Balanced); picking any sample user displays five game-recommendation rows that route through the two-lane rule from.



You also can register new User(just input username, no password required):



Appendix D: Personal Contribution

Name:	Vo Minh Khoi	**Matriculation Number:**	A0339229W

Personal contribution to group project:

This project was completed by me — I designed, built, and documented every component end-to-end, with AI assistants (ChatGPT, Gemini, Claude Chat, Claude Code, and Claude Cowork) used throughout for idea generation, debugging, illustration, and report refinement. See Appendix E for the full AI-tool usage declaration. The work falls into five groups:

- Data pipeline.
- Models and Benchmarking.
- Co-occurrence rerank and routing.
- Evaluation framework. Implemented Recall@10, NDCG@10
- Web demo and documentation**.** Built the FastAPI + SQLite + jQuery full-stack demo

What learnt is most useful for you:

- Hands-on experience training and benchmarking single-domain and cross-domain recommenders end-to-end.
- A practical understanding of how implicit feedback, user overlap, source-domain density, and catalog cleanliness drive cross-domain transfer — and which of those variables matter most.
- Familiarity with the recommender evaluation toolkit (LLO, cold-start, full-rank) and the failure modes each protocol can hide.
- Skill in turning research finding (cold-start gap, training-free co-occurrence lift) into a deployable production rule and web service.

How you can apply the knowledge and skills in other situations or your workplaces:

I want to apply these skills to real personalization, recommendation, and search systems at work. This project taught me how to handle practical issues like cold-start users, sparse data, cross-domain signals, and messy catalogues. More importantly, I learned how to design fair benchmarks, choose the right evaluation metrics, compare simple and complex methods objectively, and turn research findings into a deployable system. These are directly useful for building scalable recommender and personalization systems in industry.

Appendix E: AI Tools Usage Declaration

I declare that the following generative-AI tools were used during this project. All AI output — whether code, prose, or imagery — was reviewed, validated, and edited by me before being included in this report or in the deliverable system.

Use case	AI tools used	How it was used
Idea generation and debugging	ChatGPT, Gemini, Claude Chat	Brainstormed experimental ideas (e.g. subgroup selection, ablation designs), explored CDR algorithm intuition, and worked through model-behavior debugging and metric-anomaly investigations. All ideas are reviewed by me before being adopted to implement the project.
Coding	Claude Code, Windsurf, Cursor	Used as AI-assistant programmer/code generator for model implementations, writing benchmark tools, implementing evaluation logic, running end to end benchmarks, comparing model results and building the demo Backend/Frontend service. All produced code was reviewed and tested by me.
Visual illustrations in this report	Claude Chat, Claude Code, ChatGPT (image generation)	Generated figure drafts (PTUPCDR architecture, system architecture, routing-rule diagram, ML-pipeline diagram). Final figures were edited and re-rendered with matplotlib for consistency with the report style. ChatGPT Image was used to generate visualizations for some concepts (cold start split). All visualizations are reviewed and approved by me.
Report tooling and refinement	Google Docs Gemini, Claude Cowork	Used to generate and refine wording, tighten paragraphs, restructure tables, and splice content into the .docx (table styling, heading consistency, TOC fix-ups). All copies were reviewed and approved by me.

All algorithmic and architectural experiment decisions and conclusions in this report are mine, but with the suggestions from AI Chatbot/Agent. AI assistance accelerated execution; it did not replace judgment or verification.